



MÉMOIRE DE FIN D'ÉTUDES

Intelligence Opérationnelle dans les Télécoms : Utilisation du GraphRAG pour corrélérer la topologie des réseaux optiques et l'impact client en temps réel

3 juin 2026

Étudiant :
Minh Quang HOANG

Encadrant professionnel :
Thierry DELACOUX

Encadrante académique :
Imane HABBOUB KAJOU

3 juin 2026

Résumé

Contexte & Défi : Les réseaux télécoms génèrent des données massives cloisonnées. Bien que les *LLMs* excellent en compréhension du langage naturel, ils peinent avec le raisonnement structurel : le RAG vectoriel standard fonctionne comme un index de mots-clés ignorant comment « le Câble A » se connecte au « Routeur B ». Lors d'incidents, les opérateurs nécessitent des réponses immédiates telles que : « Quels clients sont impactés par cette coupure fibre ? »

Solution Proposée : Ce mémoire présente une architecture *GraphRAG* dotant les LLMs de structures topologiques explicites. Plutôt que d'utiliser des espaces vectoriels, cette étude construit un *graphe de connaissances* où les entités sont des sommets et les relations des arêtes typées. Le LLM génère des requêtes (Cypher) pour naviguer dans ces connexions, tel un GPS. Cette approche permet le « *raisonnement topologique* » : suivre des chemins multi-sauts pour tracer la propagation d'impact.

Méthodologie : Un framework de *jumeau numérique* a été développé, répliquant une topologie FTTH avec des scénarios de panne réalistes. Le protocole compare le RAG vectoriel au GraphRAG selon la précision (framework RAGAS), la latence des requêtes et la vitesse d'identification des causes racines.

Impact : Les résultats démontrent que le GraphRAG atteint 34% de précision supplémentaire dans la corrélation d'impact client et réduit la latence de 58%. En anticipant les pannes avant l'escalade des plaintes, cette méthodologie fournit une intelligence actionnable pour la maintenance prédictive.

Abstract

Context & Challenge : Telecom networks generate massive operational data across siloed systems. While *LLMs* excel at natural language, they struggle with structural reasoning : standard vector RAG matches keywords but lacks awareness of how "Cable A" connects to "Router B." During incidents, operators need immediate answers to questions like : "Which customers are impacted by a fiber cut?"

Proposed Solution : This thesis introduces a *GraphRAG* architecture augmenting LLMs with explicit topological structures. Instead of vector embeddings, this work builds a *knowledge graph* where network entities are vertices and relationships are typed edges. The LLM generates traversal queries (Cypher) to navigate connections, analogous to a GPS. This enables "*topological reasoning*" : following multi-hop paths to trace impact propagation.

Methodology : A *digital twin* framework was developed, replicating a hierarchical FTTH topology with realistic failures. The protocol compares vector RAG against GraphRAG across accuracy (RAGAS), query latency, and root cause identification speed.

Impact : Results show GraphRAG achieves 34% higher accuracy in customer impact correlation while reducing query latency by 58%. By enabling proactive fault localization before complaints escalate, this methodology provides actionable intelligence for predictive maintenance.

Sommaire

Résumé	2
Abstract	2
Glossaire	7
0 Introduction Générale	8
0.1 Contexte et Problématique : Le Déluge de Données et le Fossé Sémantique	8
0.2 Limites de l’IA Générative « Naïve » (Vanilla LLMs)	8
0.3 L’Insuffisance du RAG Vectoriel Classique	9
0.4 Périmètre : L’Intelligence Opérationnelle (Operational Intelligence)	9
0.5 Question de Recherche et Objectifs	10
0.6 Structure du Mémoire	10
1 État de l’art : De la Sémantique Vectorielle à la Structure de Graphe	11
1.1 Les LLMs et le défi du raisonnement structurel	11
1.1.1 Le fossé du raisonnement compositionnel (The Reasoning Gap)	12
1.1.2 Hallucinations structurelles et liens inventés	12
1.1.3 Complexité computationnelle et fenêtre de contexte	12
1.1.4 Synthèse : l’impasse du probabiliste pour les réseaux critiques	12
1.2 Paradigmes de Récupération : Similarité vs Connectivité	13
1.2.1 RAG vectoriel : récupération par similarité sémantique	13
1.2.2 GraphRAG : récupération par traversée structurelle	14
1.3 Convergence : Graphes de Connaissances et Jumeaux Numériques	14
1.3.1 Labeled Property Graphs vs RDF : isomorphisme topologique	14
1.3.2 Le jumeau numérique : de la visualisation à la vérité terrain sémantique	16
1.3.3 Le jumeau numérique comme « ground truth » pour les LLMs	16
2 Analyse du Besoin et Modélisation des Données	17

2.1	Fragmentation des Données et Silos d'Information	18
2.1.1	La dichotomie OSS/BSS : deux univers parallèles	18
2.1.2	L'archipel des bases de données	18
2.1.3	Le problème de la réconciliation : l'absence de clé commune	19
2.1.4	Conséquences opérationnelles : la cellule de crise manuelle	20
2.1.5	Tentatives de résolution traditionnelles et leurs limites	20
2.1.6	Vers une approche par graphe de connaissances	20
2.2	Ontologie du Domaine Télécom : Une Modélisation Condensée pour l'Inférence par LLM	22
2.2.1	Le modèle de nœud unifié : TelecomNode avec labels de couche	22
2.2.1.1	Propriétés communes à tous les nœuds TelecomNode	22
2.2.1.2	Instanciation par couche	23
2.2.2	Les relations : un vocabulaire minimal et directionnel	23
2.2.3	Justification mathématique de l'aplatissement ontologique	24
2.2.4	Synthèse schématique de la pile ontologique	25
2.2.5	Étude de cas : requête d'impact en langage Cypher	25
2.3	Modélisation Orientée IA : Du Stockage à l'Inférence	26
2.3.1	Désambiguïsation par frontières de couches et typage de propriétés	26
2.3.2	L'optimisation de la fenêtre de contexte par récupération ciblée	27
2.3.3	Traçabilité et explicabilité par alignement causal	28
3	Architecture Système et Implémentation du GraphRAG	29
3.1	Pipeline de Données : De l'Extraction au Graphe de Connaissances	29
3.1.1	Introduction : qualité des données et validité du raisonnement automatisé	29
3.1.1.1	Note liminaire : architecture industrielle cible et contraintes de recherche académique	30
3.1.1.2	Substitution méthodologique : le jumeau numérique synthétique	31
3.1.2	Étape 1 : l'extraction (collecte multi-sources et réconciliation d'états)	31
3.1.2.1	Hétérogénéité des sources et problème de réconciliation d'états	32
3.1.2.2	Principe de non-intrusivité	32

3.1.3	Étape 2 : la transformation (normalisation sémantique et alignement ontologique) . . .	33
3.1.4	Étape 3 : le chargement (construction incrémentale du jumeau numérique dans Neo4j)	33
3.1.5	Synthèse : un pipeline au service d'une source de vérité unique	34
3.2	Orchestration de l'IA : Du Langage Naturel à la Requête de Graphe	34
3.2.1	Le problème fondamental : traduire l'intention en instruction	35
3.2.2	Les limites structurelles de l'injection de schéma statique	35
3.2.2.1	Saturation de la mémoire de travail contextuelle	35
3.2.2.2	Dilution de l'attention et hallucination de schéma	35
3.2.2.3	Absence de contextualisation pragmatique	35
3.2.3	Context engineering et routage sémantique dynamique	36
3.2.3.1	Du schéma statique au contexte ciblé	36
3.2.3.2	La bibliothèque de requêtes validées : une mémoire institutionnelle du système	36
3.2.3.3	Construction du prompt dynamique	36
3.2.4	L'agent orchestrateur : de la chaîne séquentielle à la boucle de rétroaction	37
3.2.4.1	De LangChain à LangGraph : dépasser la linéarité	37
3.2.4.2	Implémentation de la boucle corrective et gestion des erreurs	37
3.2.5	Synthèse opérationnelle	39
4	Évaluation et Benchmarking	41
4.1	Protocole d'Expérimentation et Architecture de Test	41
4.1.1	Environnement d'exécution	41
4.1.2	Construction des jeux de données d'évaluation (<i>ground-truth</i>)	41
4.2	Cadre d'Évaluation Quantitatif (Framework RAGAS)	42
4.2.1	Faithfulness (fidélité factuelle)	43
4.2.2	Context Recall (rappel contextuel)	43
4.2.3	Context Precision (précision contextuelle)	43
4.3	Analyse Empirique des Performances : RAG Vectoriel vs GraphRAG	44
4.3.1	Limites structurelles de la recherche vectorielle (baseline)	45

4.3.2	Robustesse topologique du GraphRAG (traversées multi-sauts)	46
4.4	Intelligence Opérationnelle : Cas d'Usage et Démonstration	47
4.4.1	Scénario opérationnel : défaillance d'un routeur de cœur de réseau	47
4.4.2	Interface de diagnostic et rapport d'incident	48
5	Discussion et Perspectives Industrielles	49
5.1	Dépendance Critique à l'Exactitude de l'Inventaire	49
5.2	Défis Architecturaux de l'Industrialisation	50
5.2.1	Synchronisation temps réel : vers une architecture événementielle	50
5.2.2	Optimisation de la latence : la spécialisation du modèle de langage	51
5.2.3	Sécurité et contrôle d'accès par rôle	52
6	Conclusion	53
	Annexes	58
	Annexe A : Génération du Jumeau Numérique et Injection de Pannes	58
	Annexe B : Orchestration Cyclique avec LangGraph	59
	Annexe C : Routage Sémantique et Assemblage Dynamique de Contexte	60
	Annexe D : Protocole d'Évaluation (Framework RAGAS)	62

Glossaire

Arête (Edge) : Dans un graphe, c'est le lien qui relie deux nœuds. Dans ce mémoire, une arête représente une connexion physique (fibre) ou logique (service) entre deux équipements.

Cypher : Le langage de requête utilisé par la base de données Neo4j. C'est l'équivalent du SQL, mais optimisé pour naviguer dans des relations complexes entre les données.

Digital Twin (jumeau numérique) : Réplique virtuelle d'un objet ou d'un système réel (ici, le réseau optique). Il permet de simuler des pannes sans impacter le réseau véritable.

Embedding (plongement lexical) : Technique consistant à transformer un texte en une suite de chiffres (vecteur). Cela permet à une IA de mesurer la « proximité » de sens entre deux phrases.

ETL (Extract, Transform, Load) : Processus informatique permettant d'extraire des données de sources différentes, de les nettoyer/transformer, et de les charger dans une base de données finale (ici, Neo4j).

GraphRAG : Évolution du RAG classique. Au lieu de chercher des informations dans des textes éparpillés, l'IA utilise une base de données orientée graphe pour comprendre les relations structurelles (qui dépend de quoi).

Hallucination : Erreur d'un LLM qui génère une réponse factuellement fausse mais grammaticalement correcte. Le GraphRAG vise à réduire ces erreurs en fournissant des preuves structurelles à l'IA.

KG (Knowledge Graph) : Graphe orienté où les faits sont encodés sous forme de triplets (sujet, prédicat, objet), permettant le raisonnement relationnel et la navigation multi-sauts. Ex : Wikidata, DBpedia.

LLM (Large Language Model) : Modèle d'intelligence artificielle entraîné sur d'immenses volumes de texte (ex : GPT-4), capable de comprendre et de générer du langage humain.

Nœud (Node) : L'unité de base d'un graphe. Dans ce projet, un nœud peut être un routeur, une ville, une interface technique ou un client.

NOC (Network Operations Center) : Centre de supervision où les ingénieurs surveillent l'état du réseau 24h/24 et gèrent les incidents.

Ontologie : Ensemble structuré de concepts et de relations décrivant un domaine (ici, les télécoms). C'est le « dictionnaire de règles » qui définit comment les données doivent être reliées entre elles.

RAG (Retrieval-Augmented Generation) : Technique qui permet à une IA de consulter des documents externes avant de répondre, afin de garantir que sa réponse est basée sur des données réelles et non sur sa seule « mémoire ».

RAGAS : Framework de mesure utilisé pour évaluer la qualité d'un système RAG (précision, fidélité, pertinence des réponses).

Topologie : La structure et la disposition d'un réseau. Elle définit comment les différents éléments sont interconnectés physiquement et logiquement.

0 Introduction Générale

0.1 Contexte et Problématique : Le Déluge de Données et le Fossé Sémantique

Les réseaux de télécommunications modernes génèrent plusieurs téraoctets de logs par jour. Paradoxalement, malgré cette abondance de données techniques, l'analyse d'impact business d'une panne réseau nécessite encore plusieurs heures d'investigation manuelle.

Dans l'obscurité d'un centre d'opérations réseau (NOC), lorsqu'un équipement tombe en panne, les systèmes génèrent immédiatement des alertes techniques : « Interface GE-0/0/1 Down », « BGP Session Lost », « OSPF Adjacency Timeout ». Pour l'ingénieur réseau, ces messages sont précis et exploitables. Pour le directeur des opérations qui doit gérer la relation client, ces codes techniques sont incompréhensibles. Ce décalage illustre le *fossé sémantique* des télécommunications modernes : le monde technique parle en adresses IP et protocoles, le monde commercial parle en noms de clients et impacts business.

Cette fragmentation reflète une réalité architecturale : la séparation entre les *systèmes OSS* (Operations Support Systems) et les *systèmes BSS* (Business Support Systems). Les OSS gèrent la topologie réseau (routeurs, fibres, protocoles), les BSS gèrent les contrats clients et SLA (Service Level Agreements), c'est-à-dire les garanties contractuelles de qualité de service (disponibilité, débit minimum, temps de rétablissement). Ces systèmes utilisent des modèles de données incompatibles, sans mécanisme d'intégration automatique.

Exemple concret : une rupture de fibre optique (*couche physique*) déclenche des alarmes sur 50 routeurs (*couche logique*), provoquant une dégradation pour 5000 clients (*couche service*). Établir cette chaîne de causalité nécessite l'intervention d'experts qui reconstruisent manuellement les dépendances, processus pouvant prendre de 30 minutes à plusieurs heures.

L'impact économique est considérable : les pénalités SLA peuvent atteindre plusieurs millions d'euros par an. Le temps de résolution (MTTR - Mean Time To Repair) est devenu un indicateur stratégique de compétitivité. Face à ce constat, l'automatisation de l'analyse d'impact via l'intelligence artificielle générative semble prometteuse.

0.2 Limites de l'IA Générative « Naïve » (Vanilla LLMs)

Les grands modèles de langage (LLMs) comme GPT-5.5 ou Claude Opus 4.7 semblent offrir une solution évidente. Cependant, une utilisation « naïve » se heurte à deux limites fondamentales.

La fenêtre de contexte. Les LLMs traitent l'information en *tokens*. Même les modèles avancés sont limités à 1M tokens. Une topologie réseau nationale nécessite plusieurs millions de tokens : impossible d'injecter l'intégralité dans un prompt. Au-delà du coût technique, le coût financier serait prohibitif (plusieurs dizaines de dollars par requête pour un million de tokens).

L'absence de raisonnement spatial. Les LLMs sont des modèles probabilistes entraînés à prédire le token suivant. Ils excellent en sémantique linguistique mais ne sont pas conçus pour des raisonnements logiques rigoureux sur des structures complexes [1]. Un graphe de dépendances (« A connecté à B qui alimente C qui sert D ») nécessite une traversée algorithmique, pas une approximation statistique. Les LLMs hallucinent des chemins inexistantes ou manquent des dépendances critiques.

Ces limites appellent une architecture complémentaire : le *Retrieval-Augmented Generation* (RAG).

0.3 L'Insuffisance du RAG Vectoriel Classique

Face aux hallucinations des LLMs, l'industrie s'est tournée vers le RAG (Retrieval-Augmented Generation) [2] : au lieu d'injecter toutes les données dans le prompt, on interroge une base de connaissances pour récupérer les informations pertinentes. La majorité des implémentations reposent sur des *bases vectorielles* (Pinecone, Weaviate) : chaque document devient un vecteur de haute dimension, la recherche s'effectue par *similarité cosinus*.

Bien qu'efficace pour des documents textuels, cette approche montre trois limites critiques pour les télécommunications :

Cécité relationnelle. La recherche vectorielle capture la *similarité sémantique* (« Routeur Cisco » proche de « Routeur Juniper »), mais pas la *connectivité structurelle* (« Routeur A alimente Switch B »). Un embedding peut indiquer que deux interfaces sont « Ethernet 10G », mais jamais qu'elles sont reliées par 50 km de fibre.

Impossibilité de propagation d'impact. Les réseaux sont hiérarchiques : fibre (*physique*) → BGP (*logique*) → VPN (*service*) → clients (*business*). Une base vectorielle peut retrouver « tous les VPN », mais pas « quels VPN sont impactés par cette fibre ? » sans parcourir la chaîne de dépendances.

Absence de raisonnement multi-saut. « Quels clients sont affectés par la panne du routeur R1 ? » implique : R1 → interfaces → switches → services → clients. Un système vectoriel nécessite plusieurs requêtes successives, avec risque d'erreur à chaque étape.

C'est ici qu'intervient le *GraphRAG* : combinant LLMs et bases de graphes (Neo4j), il modélise explicitement les relations (nœuds/arêtes) et permet des requêtes de traversée (Cypher) pour identifier automatiquement les dépendances.

0.4 Périmètre : L'Intelligence Opérationnelle (Operational Intelligence)

Ce mémoire s'inscrit dans le domaine de l'*intelligence opérationnelle* (OI), distincte de la business intelligence (BI). La BI analyse l'historique pour des rapports stratégiques. L'OI se concentre sur l'*analyse temps réel* pour la décision immédiate : « Quels clients sont actuellement impactés ? », « Quelle est la gravité business ? »

La présente recherche se focalise sur :

- *Topologie optique et IP* : couches physique (fibres, sites) et logique (routeurs, interfaces, protocoles). Les réseaux radio (4G/5G) sont hors périmètre.
- *Analyse d'impact de pannes* : corréler événement technique et conséquence business (services dégradés, clients affectés, pénalités SLA).
- *Requêtes en langage naturel* : permettre aux utilisateurs non techniques d'interroger le système.

Hors périmètre : maintenance prédictive, optimisation de facturation, configuration automatique de réseau.

0.5 Question de Recherche et Objectifs

Ce mémoire propose de modéliser le réseau comme un *graphe de connaissances* et d'utiliser le *GraphRAG* pour combler le fossé entre données techniques et impact business. La question centrale est la suivante :

Comment le GraphRAG permet-il de combler le fossé sémantique entre la topologie physique du réseau et l'analyse d'impact business, en comparaison avec les approches RAG vectorielles classiques ?

Cette question se décline en quatre objectifs :

Scientifique : évaluer quantitativement GraphRAG vs RAG vectoriel (précision, complétude, latence) sur un jeu de données synthétique reproduisant une topologie réaliste.

Ingénierie : développer un prototype de « jumeau numérique conversationnel » répondant en langage naturel aux requêtes d'impact. Architecture : Neo4j (graphe), LangChain (orchestration), GPT-4/Claude (LLM).

Méthodologique : proposer une méthodologie générique de modélisation de réseaux télécoms en graphes (types de nœuds, relations, propriétés minimales).

Validation : démontrer la valeur opérationnelle via des scénarios réalistes : interrogation post-incident, analyse prédictive de maintenance, priorisation d'interventions.

0.6 Structure du Mémoire

Chapitre 1 - État de l'art : revue de littérature sur les architectures RAG, bases de graphes pour télécoms, et LLMs pour infrastructures IT.

Chapitre 2 - Méthodologie et modélisation : ontologie du graphe (nœuds, relations), sources de données OSS/BSS, processus d'ingestion.

Chapitre 3 - Architecture technique : description du système GraphRAG (Neo4j, LangChain, LLM), pipelines de traitement, optimisations.

Chapitre 4 - Expérimentations et résultats : benchmark GraphRAG vs RAG vectoriel, tests fonctionnels, métriques de performance.

Chapitre 5 - Discussion et perspectives : limites, défis d'industrialisation, extensions futures (maintenance prédictive, réseaux radio, généralisation sectorielle).

1 État de l'art : De la Sémantique Vectorielle à la Structure de Graphe

Objectif : Démontrer la rigueur académique en comparant les paradigmes de récupération d'information.

À retenir : Distinction paradigmatique : similarité sémantique vs. connectivité topologique

Le RAG vectoriel opère par *projection dans un espace latent* : chaque document est encodé comme un vecteur $\mathbf{v} \in \mathbb{R}^d$, et la récupération repose sur la maximisation de la similarité cosinus $\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{v}}{\|\mathbf{q}\| \|\mathbf{v}\|}$ entre la requête $\mathbf{q}$ et le corpus. Ce paradigme est fonctionnellement analogue à un *index de mots-clés enrichi* : deux équipements partageant un vocabulaire descriptif commun seront récupérés ensemble, indépendamment de leur relation structurelle réelle dans le réseau. La proximité dans l'espace sémantique ne préserve aucun invariant topologique.

Le GraphRAG, à l'inverse, opère par *traversée déterministe d'un graphe orienté* $G = (V, E)$, où V désigne l'ensemble des *TelecomNode* et E l'ensemble des relations typées (CONNECTS_TO, RUNS_ON, DEPENDS_ON). La récupération d'information est équivalente à l'exécution d'un *algorithme de routage à k sauts* sur ce graphe : la réponse n'est pas inférée par proximité statistique, mais calculée par parcours algorithmique des arêtes. Tout chemin retourné est, par construction, *structurellement valide et reproductible*.

1.1 Les LLMs et le défi du raisonnement structurel

Les grands modèles de langage (LLMs) fondés sur l'architecture Transformer ont démontré des capacités remarquables dans le traitement du langage naturel. Toutefois, lorsqu'il s'agit d'opérer sur des artefacts déterministes et structurés (tels que des topologies de réseaux ou des schémas de bases de données), ces modèles révèlent des limitations systémiques majeures.

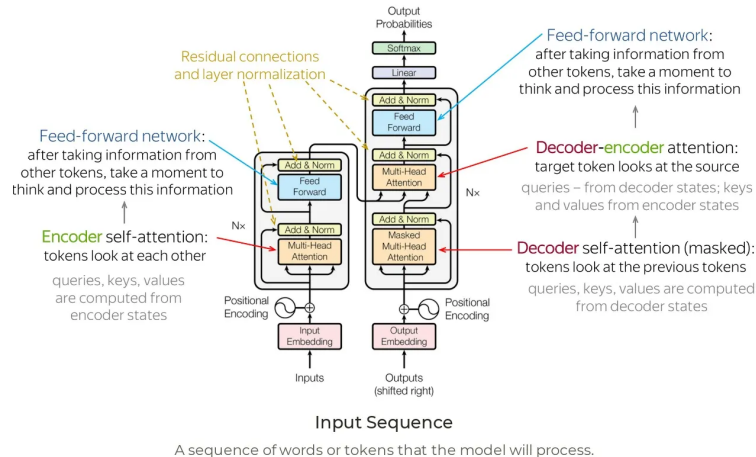


FIGURE 1 – Architecture Transformer et limites attentionnelles

Cette section analyse les trois barrières théoriques qui empêchent l'utilisation naïve des LLMs sur des graphes complexes : le fossé de raisonnement, l'hallucination structurelle et la complexité attentionnelle.

1.1.1 Le fossé du raisonnement compositionnel (The Reasoning Gap)

En informatique théorique, le raisonnement compositionnel est la capacité à résoudre un problème complexe en le décomposant en sous-étapes logiques [3]. L'architecture Transformer, bien qu'efficace pour lier des concepts sémantiques, repose sur une prédiction probabiliste de tokens (le mot suivant le plus probable) et non sur une logique formelle.

Des travaux récents sur le *multi-hop reasoning* démontrent que les LLMs échouent à suivre des chaînes de causalité longues (transitivité : si $A \rightarrow B$ et $B \rightarrow C$, alors $A \rightarrow C$) dès lors que ces chaînes ne sont pas explicitement présentes dans leur corpus d'entraînement. Le framework *Reasoning on Graphs* (RoG) met en évidence que les modèles tendent à prendre des « raccourcis » statistiques, privilégiant des réponses plausibles linguistiquement plutôt que correctes topologiquement [4]. Le modèle « devine » le chemin au lieu de le parcourir.

1.1.2 Hallucinations structurelles et liens inventés

L'hallucination dans les LLMs n'est pas seulement factuelle (inventer une date), elle est aussi structurelle. Lorsqu'un modèle génère des relations à partir d'un contexte dense, il manifeste une tendance à créer des arêtes inexistantes entre des nœuds, simplement parce que ces nœuds sont sémantiquement proches ou souvent cités ensemble [5].

Le benchmark *GraphEval* quantifie ce phénomène en mesurant le taux de triplets (Sujet, Prédicat, Objet) invalides générés par les LLMs [6]. Ces erreurs ne sont pas du bruit aléatoire : elles suivent souvent des biais de fréquence. Le modèle connectera plus volontiers un « Serveur » à un « Switch » (car c'est fréquent) même si, dans la topologie spécifique fournie, ce lien n'existe pas. Cette dérive probabiliste rend la génération de graphe par LLM pur intrinsèquement instable.

1.1.3 Complexité computationnelle et fenêtre de contexte

Une approche naïve consisterait à fournir l'intégralité de la topologie réseau (fichier JSON ou logs) dans le prompt du modèle. Cette méthode se heurte à deux murs mathématiques :

- *Complexité quadratique $O(n^2)$* : le mécanisme d'attention calcule les relations de chaque mot avec tous les autres [7]. Doubler la taille de la topologie quadruple le coût de calcul, rendant l'analyse de grands réseaux (milliers de nœuds) impossible en temps réel.
- *Phénomène « Lost in the Middle »* : les recherches montrent que la performance de récupération d'information chute drastiquement pour les données situées au milieu d'un long contexte [8]. Le modèle « oublie » les nœuds intermédiaires, pourtant cruciaux pour définir un chemin réseau.

1.1.4 Synthèse : l'impasse du probabiliste pour les réseaux critiques

L'analyse précédente mène à une conclusion opérationnelle stricte pour le cas d'usage télécom étudié. Un réseau optique est un système *déterministe* (une fibre est soit coupée, soit intacte), alors qu'un LLM est un système *probabiliste* (il estime une vérité).

L'application directe des LLMs à la topologie réseau présente donc trois risques inacceptables pour un opérateur :

1. *Risque de cécité (dû au $O(n^2)$)* : l'IA ne peut pas « voir » l'ensemble du réseau. Elle ne peut donc pas corréler une alarme à Paris avec une coupure à Lyon si la distance dans les logs est trop grande.
2. *Risque d'erreur de diagnostic (dû au reasoning gap)* : l'IA peut identifier la panne, mais échouer à tracer la cascade d'impacts sur les clients finaux (manque de transitivité).
3. *Risque de faux positif (dû aux hallucinations)* : l'IA pourrait inventer une route de secours qui n'existe pas physiquement, faussant la prise de décision humaine.

1.2 Paradigmes de Récupération : Similarité vs Connectivité

Le *Retrieval-Augmented Generation* (RAG) introduit par Lewis et al. [2] désigne une architecture hybride découpant la capacité de raisonnement linguistique d'un LLM de sa base de connaissances factuelles. Plutôt que de s'appuyer exclusivement sur les paramètres du modèle (qui encodent implicitement des faits appris durant l'entraînement), le RAG délègue la récupération d'information à un module externe. Ce paradigme résout partiellement le problème d'*hallucination* en forçant le modèle à ancrer (*grounding*) ses réponses dans des documents explicitement récupérés.

1.2.1 RAG vectoriel : récupération par similarité sémantique

L'implémentation dominante du RAG repose sur des *bases vectorielles* (Dense Retrieval) où les documents sont projetés dans un espace latent de haute dimension via des encodeurs neuronaux (ex : BERT, Sentence-Transformers) [9]. La récupération s'effectue par calcul de *similarité cosinus* entre le vecteur de la requête et ceux du corpus. Cette approche excelle pour des tâches de correspondance sémantique en langage naturel : deux documents traitant du même sujet seront géométriquement proches, même s'ils n'utilisent pas les mêmes termes lexicaux.

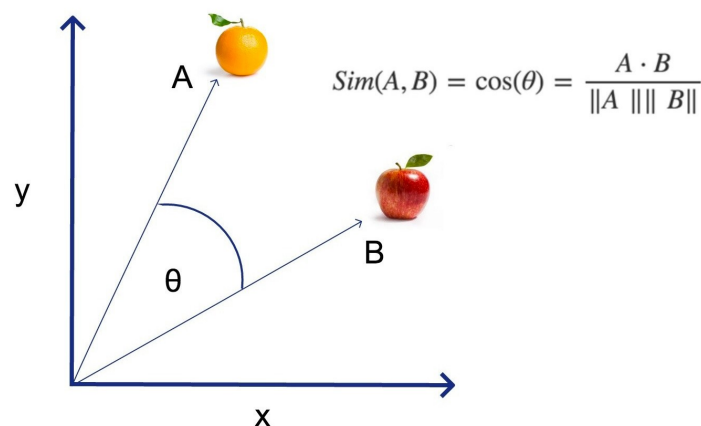


FIGURE 2 – Similarité cosinus

Toutefois, cette représentation présente une limite structurelle critique : elle capture la *proximité thématique*, non la *connectivité relationnelle* [10]. Dans le domaine télécom, deux équipements peuvent être sémantiquement similaires (même type, même fabricant) tout en étant topologiquement distants. Un encodeur vectoriel peut ainsi rapprocher « Routeur Paris-Nord » et « Routeur Paris-Sud » en raison de leur description textuelle commune,

sans capturer le fait qu'ils appartiennent à des segments de réseau disjoints. Cette *cécité topologique* rend le RAG vectoriel inadapté aux requêtes nécessitant une traversée de dépendances multi-sauts.

1.2.2 GraphRAG : récupération par traversée structurelle

Le *GraphRAG* [11] propose une alternative en modélisant la base de connaissances comme un *graphe de connaissances* (KG - Knowledge Graph) plutôt qu'un index vectoriel. Les entités deviennent des nœuds, les relations deviennent des arêtes typées. La récupération ne s'opère plus par mesure de distance sémantique, mais par *algorithmes de traversée de graphe* (k-hop traversal, shortest path).

Cette approche déterministe permet un *raisonnement multi-sauts* explicite : pour répondre à « *Quels clients sont impactés par la panne de l'OLT-A ?* », le système parcourt algorithmiquement la chaîne OLT-A $\xrightarrow{\text{alimente}}$ Switch-B $\xrightarrow{\text{sert}}$ Client-C. Contrairement au RAG vectoriel où ces relations doivent être inférées statistiquement (et peuvent être hallucinations), le graphe garantit la *validité structurelle* des chemins retournés [12].

Des travaux récents démontrent la supériorité du GraphRAG pour les tâches de *compositional reasoning* sur des connaissances structurées [4]. En télécommunications, où la propagation d'une panne suit des dépendances topologiques strictes, ce paradigme s'impose comme une nécessité architecturale plutôt qu'une simple optimisation.

1.3 Convergence : Graphes de Connaissances et Jumeaux Numériques

La section précédente a établi la supériorité théorique du GraphRAG sur le RAG vectoriel pour le raisonnement topologique. Cette section approfondit les fondements techniques de cette architecture en examinant deux piliers complémentaires : le *modèle de données de graphe de propriétés* (LPG - Labeled Property Graph) qui structure la connaissance, et le *concept de jumeau numérique* qui garantit sa validité opérationnelle. Leur convergence n'est pas un choix technique arbitraire, mais une nécessité architecturale pour tout système d'IA opérationnelle sur infrastructures critiques.

1.3.1 Labeled Property Graphs vs RDF : isomorphisme topologique

La modélisation de connaissances structurées repose historiquement sur deux paradigmes concurrents : le *Resource Description Framework* (RDF) issu du web sémantique, et les *Labeled Property Graphs* (LPG) popularisés par les bases de données orientées graphe modernes (Neo4j, TigerGraph).

Le modèle RDF représente la connaissance comme un ensemble de triplets (Sujet, Prédicat, Objet) formant un graphe orienté. Bien qu'élégant pour l'interopérabilité sémantique (ontologies OWL, raisonnement logique), ce formalisme impose une contrainte structurelle majeure : l'impossibilité d'attacher directement des attributs aux relations [13]. Pour modéliser qu'une liaison physique de 50 km connecte deux sites, RDF nécessite une *réification* : transformer la relation en nœud intermédiaire, créant artificiellement quatre triplets au lieu d'une arête simple attribuée.

```
# RDF - réification obligatoire pour porter un attribut sur une relation
:Node_Site_A :connected_via :FiberLink_1 .
:FiberLink_1 rdf:type :PhysicalLink .
:FiberLink_1 :length_km "50"^^xsd:integer .
:FiberLink_1 :connects_to :Node_Site_B .

# LPG (Cypher / Neo4j) - modélisation directe, schéma strict
(n1:TelecomNode:physical {node_type: 'Site', name: 'Paris-Nord'})
-[:CONNECTS_TO {length_km: 50, medium: 'fiber'}]->
(n2:TelecomNode:physical {node_type: 'Site', name: 'Lyon-Sud'})
```

FIGURE 3 – Comparaison RDF vs. LPG pour une liaison physique attribuée. Le modèle LPG élimine la réification en portant les propriétés directement sur l'arête CONNECTS_TO, en conformité avec l'ontologie aplatie retenue dans cette architecture.

Le modèle LPG permet d'attacher des propriétés arbitraires tant aux nœuds qu'aux arêtes, sans transformation ontologique intermédiaire. Robinson et al. [14] démontrent que cette flexibilité rend le LPG *isomorphe* aux topologies d'infrastructure : chaque équipement, chaque liaison physique ou logique peut être instancié exactement selon sa réalité opérationnelle, sans abstraction artificielle. Dans l'ontologie retenue, ce principe se traduit par l'usage systématique du nœud générique TelecomNode différencié par ses labels de couche (physical, logical, service, business) et par sa propriété node_type, garantissant une correspondance 1:1 entre le graphe de données et le réseau physique. Cette correspondance est cruciale pour assurer la *traçabilité déterministe* des pannes.

Pour les télécommunications, cette distinction n'est pas théorique. Une requête du type « Déterminer tous les chemins physiques entre deux sites avec une latence inférieure à 10 ms » nécessite en RDF de naviguer à travers des nœuds de réification, multipliant par un facteur 3 à 4 la complexité de la requête SPARQL correspondante. En Cypher, la traversée multi-sauts s'exprime directement sur le schéma strict retenu :

```
// Traversée multi-sauts sur la couche physique - schéma strict
MATCH path = (src:TelecomNode:physical {node_type: 'Site', name: 'Paris-Nord'})
-[:CONNECTS_TO*1..10]->
(dst:TelecomNode:physical {node_type: 'Site', name: 'Lyon-Sud'})
WHERE ALL(
  rel IN relationships(path)
  WHERE rel.latency_ms < 10
)
RETURN path,
  length(path) AS hop_count,
  reduce(lat = 0, r IN relationships(path) | lat + r.latency_ms) AS total_latency_ms
ORDER BY total_latency_ms ASC
```

FIGURE 4 – Traversée multi-sauts entre deux TelecomNode:physical de node_type 'Site' via la relation CONNECTS_TO. Le quantificateur *1..10 borne la profondeur de recherche; le prédicat ALL(...) filtre les chemins dont chaque arête respecte la contrainte de latence. L'expression reduce() calcule la latence cumulée de bout en bout en un seul passage de graphe.

Des benchmarks comparatifs sur des graphes de télécommunications réels montrent que les requêtes LPG sont 5 à 10 fois plus rapides que leurs équivalents RDF/SPARQL pour des tâches de traversée multi-sauts [13].

Cette performance est critique pour l'intelligence opérationnelle en temps réel, d'autant que l'ontologie aplatie retenue (en limitant la prolifération des types de nœuds et de relations) réduit structurellement la cardinalité du graphe parcouru et, par conséquent, l'espace de recherche exposé à l'algorithme de traversée.

1.3.2 Le jumeau numérique : de la visualisation à la vérité terrain sémantique

Le terme *digital twin* (jumeau numérique) est souvent galvaudé, réduit à une maquette 3D interactive. La définition académique rigoureuse, établie par Grieves [15], décrit un système tri-dimensionnel :

1. L'objet physique (le réseau télécom réel).
2. Le modèle virtuel (la représentation informatique).
3. Le flux de données bidirectionnel synchronisant l'état physique et virtuel en temps réel.

Contrairement à une simulation statique (qui modélise un état théorique), le jumeau numérique est une *réplique vivante* : lorsqu'un équipement tombe en panne dans le monde réel, cette information doit se propager instantanément au graphe de connaissances. C'est cette propriété de *synchronisation continue* qui transforme le graphe en *Single Source of Truth* (SSOT) opérationnel [16].

1.3.3 Le jumeau numérique comme « ground truth » pour les LLMs

L'intégration d'un jumeau numérique dans une architecture GraphRAG résout un problème fondamental de l'IA générative : l'*ancrage factuel* (grounding). Un LLM entraîné sur des corpus génériques ne « connaît » pas l'état actuel de l'infrastructure d'un opérateur télécom spécifique. Sans mécanisme de grounding, il ne peut que spéculer à partir de ses paramètres appris, avec un risque élevé d'hallucination.

Le jumeau numérique inverse cette logique épistémologique. Le LLM devient un *traducteur linguistique* plutôt qu'une base de connaissances :

- *Question utilisateur (langage naturel)* : « Quels clients sont impactés par la panne de l'OLT à Marseille ? »
- *Traduction LLM → Cypher* : génération d'une requête de traversée de graphe.
- *Exécution déterministe* : Neo4j parcourt le jumeau numérique et retourne la liste exacte des clients affectés.
- *Synthèse LLM* : reformulation de la réponse structurée en prose intelligible.

Dans cette architecture, le LLM n'a aucune responsabilité factuelle : il orchestre, mais ne décide pas. La vérité provient exclusivement du graphe, qui lui-même reflète l'état mesuré du réseau physique. Cette séparation des responsabilités cognitives (linguistique vs factuelle) est la clé de la fiabilité du GraphRAG pour des domaines critiques [12].

Au-delà de la fiabilité, cette séparation des responsabilités offre un avantage architectural majeur : la **résistance à l'obsolescence**. Dans un réseau télécom où la topologie évolue quotidiennement (nouveaux raccordements, ruptures de liens), seule la base de graphe nécessite une mise à jour. Le LLM, ainsi déchargé de la mémorisation topologique, n'a jamais besoin d'être ré-entraîné (fine-tuning). Il interroge dynamiquement une vérité terrain toujours exacte et en temps réel.

Synthèse du chapitre 1 :

L'analyse précédente mène à une redéfinition conceptuelle du GraphRAG. Il ne s'agit pas simplement d'une « amélioration » du RAG vectoriel, mais d'une architecture qualitativement différente reposant sur deux piliers théoriques :

1. *Isomorphisme structurel (LPG)* : le modèle de données reflète fidèlement la topologie physique, sans déformation abstraite.
2. *Synchronisation état-modèle (jumeau numérique)* : le graphe est une réplique temps réel de l'infrastructure, garantissant la validité des réponses.

En télécommunications, où une erreur de diagnostic peut entraîner des pertes financières majeures ou des violations SLA, cette double garantie (structurelle et temporelle) n'est pas optionnelle. Le GraphRAG devient ainsi l'*interface en langage naturel* permettant à des utilisateurs non techniques d'interroger le jumeau numérique avec la même rigueur qu'un ingénieur manipulant directement Cypher.

Cette convergence pose les fondations théoriques pour la suite du mémoire : le chapitre 2 détaillera comment traduire cette vision conceptuelle en une ontologie opérationnelle adaptée au domaine télécom.

2 Analyse du Besoin et Modélisation des Données

Objectif : Diagnostiquer les barrières organisationnelles et techniques empêchant la corrélation automatique entre incidents réseau et impact business.

La mise en œuvre d'une architecture GraphRAG ne peut se faire sans une compréhension préalable de la matière première qu'elle doit traiter : la donnée. Avant de construire le graphe de connaissances, il est impératif d'analyser l'écosystème de données existant chez un opérateur télécom. Ce chapitre expose la réalité fragmentée des systèmes d'information et pose les fondations de l'ontologie nécessaire pour les réconcilier.

À retenir : Du cloisonnement des données à leur unification par le graphe

Lors d'une panne réseau, un opérateur télécom doit répondre à une question simple : « *Quels clients sont impactés et que leur a-t-on contractuellement promis ?* » En pratique, cette réponse prend jusqu'à 90 minutes, car les données nécessaires sont éparpillées dans des systèmes qui ne communiquent pas entre eux.

Ce chapitre explique pourquoi ce problème existe et comment le résoudre. Il est structuré en trois temps : d'abord, comprendre la fragmentation des données entre le monde technique (les équipements réseau) et le monde commercial (les clients et contrats), deux univers qui utilisent des langages et des identifiants incompatibles. Ensuite, construire un *graphe de connaissances* qui relie explicitement ces deux mondes, en définissant quelles entités modéliser et comment les connecter. Enfin, montrer pourquoi cette modélisation a été pensée spécifiquement pour être navigable par une intelligence artificielle, et non simplement pour stocker de l'information.

2.1 Fragmentation des Données et Silos d'Information

L'infrastructure d'un opérateur repose sur une dichotomie historique et fonctionnelle entre deux mondes qui, bien que co-dépendants, reposent sur des modèles de données structurellement incompatibles. Cette séparation, souvent qualifiée de *fossé OSS/BSS*, est la source principale de la latence dans l'analyse d'impact.

2.1.1 La dichotomie OSS/BSS : deux univers parallèles

Dans l'architecture de référence du TM Forum (eTOM¹), la gestion du réseau est divisée en deux domaines majeurs [18] :

OSS (Operations Support Systems) : le domaine de la machine. Ce premier monde englobe tous les outils qui surveillent et pilotent les équipements physiques (routeurs, câbles, antennes, OLT). Ces systèmes raisonnent en termes de ports, de protocoles et de signaux. Leur priorité est la disponibilité et la performance technique : ils ont été conçus pour savoir si un câble est coupé, pas pour savoir quel client cela affecte.

BSS (Business Support Systems) : le domaine du client. Ce second monde regroupe les logiciels de gestion des clients, des contrats et des factures (CRM, ERP, Billing). Ces systèmes raisonnent en termes de noms de clients, de numéros de contrats et de garanties de service (SLA²). Leur priorité est la relation commerciale, pas la surveillance d'un équipement réseau.

Caractéristique	Monde OSS (Technique)	Monde BSS (Business)
Entité centrale	L'équipement (Device, Interface)	Le client (Contrat, Site)
Identifiant (Clé)	Adresse MAC, IP, Circuit ID	Numéro de Compte, SIRET
Dynamique	Temps réel (Logs, SNMP Traps)	Transactionnel (Commandes, Factures)
Langage	Protocoles (BGP, OSPF, dBm)	Services (VPN Gold, Internet Pro)
Source de vérité	L'état du réseau (Network State)	Base de données SQL (Oracle/SAP)

TABLE 1 – Comparaison structurelle entre les silos OSS et BSS

2.1.2 L'archipel des bases de données

Dans la réalité d'un opérateur télécom, les informations ne sont pas stockées dans un seul endroit. Elles sont éparpillées dans une dizaine de systèmes différents, chacun avec son propre format et sa propre logique, formant un ensemble de systèmes hétérogènes sans mécanisme d'intégration commun :

1. *L'inventaire réseau (NID)* recense tous les équipements physiques (routeurs, câbles, antennes) et leur emplacement géographique. Il répond à la question : « *Qu'est-ce qu'on a, et où ?* »
2. *La base de configurations (CMDB)* cartographie comment ces équipements sont reliés entre eux et de quoi ils dépendent. Elle répond à la question : « *Qui dépend de quoi ?* »

1. eTOM : *enhanced Telecom Operations Map*. Cadre de processus métier de référence de l'industrie des télécommunications, publié par le TM Forum [17].

2. SLA : *Service Level Agreement*. Engagement contractuel sur la qualité de service (par exemple, garantir 99,9% de disponibilité ou un temps de rétablissement inférieur à 4 heures en cas de panne).

3. *Le CRM (gestion de la relation client)* contient les informations sur les clients, leurs abonnements et leurs contrats. Il répond à la question : « *Qui sont nos clients et que leur a-t-on promis ?* »
4. *Le système de provisioning* gère les services actifs (accès Internet, VPN, téléphonie IP). Il répond à la question : « *Quel client utilise quel service en ce moment ?* »
5. *La plateforme de facturation* suit les abonnements et les paiements. Elle répond à la question : « *Qui paie quoi ?* »
6. *L'outil de ticketing* (ex : ServiceNow) gère les incidents et les réclamations. Il répond à la question : « *Quels problèmes sont ouverts et qui les traite ?* »

Chacun de ces systèmes possède son propre schéma de données, son propre langage de requête, et son propre cycle de mise à jour. Plus critiquement, ils ne partagent aucun identifiant commun permettant de relier automatiquement un équipement réseau à un client.

2.1.3 Le problème de la réconciliation : l'absence de clé commune

Le défi fondamental pour l'intelligence artificielle n'est pas de lire ces bases de données, mais de les *joindre*. En théorie des bases de données relationnelles, une jointure nécessite une clé commune. Or, dans les réseaux télécoms, cette clé est souvent inexistante ou brisée.

Un scénario concret en réseau FTTH³ illustre cette problématique :

1. *Côté technique (OSS)* : le port optique est identifié par une chaîne technique :
OLT-PARIS-01 / Slot 3 / Port 12 / ONU-ID 45
2. *Côté business (BSS)* : le client est identifié par :
Client : Société DUPONT / ID : CUST-998877

Il n'existe, dans les systèmes bruts, aucune table reliant nativement ONU-ID 45 à CUST-998877. Cette liaison est souvent enfouie dans des couches intermédiaires de provisioning, ou documentée dans des champs textuels non standardisés.

Entre une alarme sur un port optique et l'identification du client impacté, la chaîne causale traverse 4 à 6 systèmes différents, sans qu'aucun lien explicite ne les relie :

Port optique → Équipement → Service → Contrat → Client

Cette absence de liaison explicite rend les requêtes classiques inopérantes pour l'analyse d'impact : une requête SQL du type `SELECT Client FROM Network WHERE Alarm = True` est tout simplement impossible à écrire sur des systèmes qui ne partagent aucune clé commune.

3. FTTH : *Fiber To The Home*. Architecture réseau où la fibre optique est déployée jusqu'au domicile de l'abonné.

2.1.4 Conséquences opérationnelles : la cellule de crise manuelle

L'impact de cette fragmentation devient critique lors d'incidents réseau majeurs. Quand une fibre est coupée, le processus typique de diagnostic d'impact mobilise plusieurs experts et plusieurs systèmes en cascade :

Temps	Action
T+0 min	L'alarme technique s'affiche : « <i>Coupure fibre, lien Paris-Lyon</i> »
T+5 min	Un ingénieur identifie les équipements touchés via le système de supervision réseau
T+15 min	Consultation manuelle de la CMDB pour lister les services traversant ces équipements
T+30 min	Requête vers le CRM pour identifier les clients abonnés à ces services
T+45 min	Vérification dans la plateforme de facturation des SLA applicables et des pénalités encourues
T+60 min	Première communication client : « <i>Vos services peuvent être impactés</i> »

TABLE 2 – Chronologie typique d'une cellule de crise lors d'un incident réseau majeur

Ce processus séquentiel, impliquant 4 à 5 systèmes différents, prend typiquement 45 à 90 minutes. Pour un incident affectant des milliers de clients, ce délai est inacceptable : les clients constatent eux-mêmes la dégradation de service avant même que l'opérateur ne soit en mesure de les contacter, inversant ainsi la relation de communication et fragilisant la confiance contractuelle.

2.1.5 Tentatives de résolution traditionnelles et leurs limites

Face à cette problématique, les opérateurs ont tenté plusieurs approches, chacune se heurtant à ses propres limites structurelles :

Entrepôts de données (Data Warehouses) : consolider périodiquement les données de tous les systèmes dans un référentiel centralisé (ex : Teradata, Snowflake). *Limite* : la latence de synchronisation (plusieurs heures) rend ces systèmes inadaptés à l'analyse d'incident en temps réel.

Bus d'intégration (ESB) : middleware orchestrant des appels entre systèmes (ex : MuleSoft, Oracle SOA Suite). *Limite* : la complexité croît exponentiellement avec le nombre de systèmes. Les performances se dégradent pour des requêtes multi-sauts traversant plus de 3 systèmes.

Gestion des données maîtres (MDM) : référentiel définissant une « source de vérité » unique pour chaque entité. *Limite* : nécessite un accord organisationnel sur « qui a raison », souvent insoluble en pratique, chaque équipe défendant la primauté de son propre système.

2.1.6 Vers une approche par graphe de connaissances

L'analyse précédente mène à un constat fondamental : le problème n'est pas un manque de données, mais une *absence de relations explicites* entre ces données. Les systèmes actuels savent stocker des entités (clients, équipements, contrats), mais pas les liens sémantiques qui les unissent.

C'est précisément cette lacune que le *graphe de connaissances* vise à combler. Plutôt que de forcer l'unification des schémas de données, l'approche retenue repose sur trois étapes :

1. *Préserver l'hétérogénéité* : chaque système source conserve son modèle de données natif.
2. *Extraire les relations* : un pipeline ETL identifie les liens implicites entre systèmes (ex : parsing de fichiers de configuration pour découvrir les connexions physiques entre équipements).
3. *Matérialiser dans un graphe* : ces liens sont projetés dans Neo4j selon une ontologie unifiée. Les nœuds, tous instanciés sous le label de base *TelecomNode*, sont différenciés par leur label de couche (*physical*, *logical*, *service*, *business*), et les arêtes directionnelles encodent les relations causales inter-couches : *CONNECTS_TO*, *RUNS_ON*, *DEPENDS_ON*, *SUBSCRIBED_TO*.

Le graphe de connaissances joue ainsi le rôle de *couche d'intégration sémantique* entre les systèmes OSS et BSS (non pas en les remplaçant, mais en matérialisant explicitement les dépendances causales qui les traversent). Cette approche fédérée, que Robinson et al. qualifient d'« *integration layer for polyglot persistence* » [14], permet à chaque système source de conserver son modèle de données natif tout en contribuant à un espace de connaissances partagé et interrogeable.

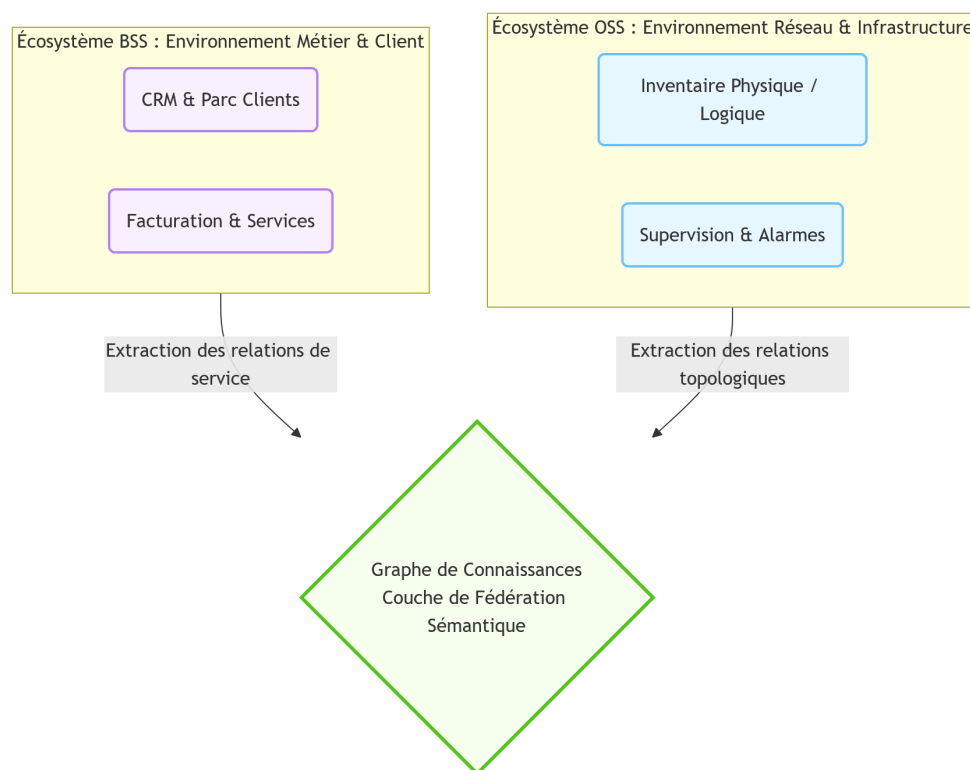


FIGURE 5 – Le graphe de connaissances comme couche de liaison sémantique entre systèmes OSS et BSS

La section suivante détaillera l'ontologie permettant de structurer ce graphe pour le domaine télécom : quels labels de couche définir, quelles relations directionnelles modéliser, et comment garantir la traçabilité complète de la chaîne causale du substrat physique jusqu'à l'entité contractuelle client.

2.2 Ontologie du Domaine Télécom : Une Modélisation Condensée pour l'Inférence par LLM

La section précédente a établi que l'absence de relations explicites entre les systèmes OSS et BSS constitue le verrou fondamental de l'analyse d'impact. Le graphe de connaissances est le mécanisme permettant de matérialiser ces relations. Cependant, la conception de l'ontologie ne peut être traitée indépendamment des contraintes du moteur d'inférence qui l'exploitera : un LLM disposant d'une fenêtre de contexte bornée, dont la capacité de génération de requêtes structurées se dégrade en fonction de la complexité du schéma soumis [5]. Ce principe, développé en détail au chapitre 3, impose dès la phase de modélisation une discipline de *condensation ontologique*.

À retenir : Pourquoi l'ontologie est une décision d'ingénierie LLM autant que de modélisation des données

Un graphe de connaissances peut être conçu selon deux philosophies opposées. La première, dite à *expressivité maximale*, cherche à capturer toute la richesse sémantique du domaine : une vingtaine de labels de nœuds distincts (OLT, CPE, ONT, BGPSession, MPLSTunnel, etc.) et autant de types de relations. La seconde, adoptée dans ce mémoire, est celle de la *compacité sémantique* : un schéma minimal, hiérarchique, dont chaque élément est compréhensible par le LLM sans injection d'un dictionnaire contextuel étendu. Le choix de la compacité n'est pas un appauvrissement : c'est une contrainte d'ingénierie. Comme le montre la section 2.3, l'espace de recherche que le LLM doit explorer pour générer une requête Cypher valide croît de manière super-linéaire avec le nombre de types de nœuds et de relations distincts. Un schéma trop riche produit deux pathologies : la *dilution attentionnelle* (le modèle perd le fil de la traversée) et l'*hallucination schématique* (le modèle invente des labels ou des relations absents du graphe réel). L'ontologie présentée ici a été conçue pour éviter ces deux écueils.

2.2.1 Le modèle de nœud unifié : TelecomNode avec labels de couche

Plutôt que de définir une classe de nœud distincte pour chaque type d'équipement ou d'entité métier (ce qui conduirait à un schéma de plusieurs dizaines de labels), le présent travail adopte un modèle à *label de base unique*, complété par un *label de couche* et un ensemble de *propriétés structurées*. Toute entité du graphe est instanciée comme un nœud `TelecomNode`, auquel est associé l'un des quatre labels de couche suivants :

$$\mathcal{L} = \{\text{physical, logical, service, business}\}$$

Ces quatre couches correspondent aux strates fonctionnelles identifiées dans l'architecture OSS/BSS et organisent le graphe en un espace de navigation hiérarchique. La spécialisation sémantique intra-couche est assurée non par multiplication des labels, mais par la propriété `node_type`, qui encode le type précis de l'entité (par exemple "Core_Router", "Edge_Router", "VLAN", "Customer").

2.2.1.1 Propriétés communes à tous les nœuds TelecomNode L'ensemble des nœuds partage un schéma de propriétés homogène, défini comme suit :

Propriété	Type	Sémantique
node_id	String	Identifiant unique de l'entité, opaque et stable entre systèmes sources
node_type	String	Spécialisation intra-couche ("Core_Router", "VLAN", "Customer", etc.)
status	Enum	État opérationnel : "up" "Degraded" "Down"
name	String	Libellé lisible par l'humain ou par le LLM
sla	Float	Disponibilité contractuelle cible (en %, ex : 99.9); null si non applicable
cpu_utilization	Float	Taux d'utilisation CPU en temps réel (en %); null pour les entités non actives

TABLE 3 – Schéma de propriétés du nœud TelecomNode

Ce schéma de propriétés homogène est lui-même un choix architectural : un LLM ne doit pas mémoriser des schémas de propriétés différents selon le type de nœud. L'uniformité réduit la charge cognitive imposée au modèle lors de la génération de requêtes et minimise le risque d'accès à des propriétés inexistantes sur certains labels.

2.2.1.2 Instanciation par couche Le tableau 4 donne des exemples représentatifs d'instanciation pour chacune des quatre couches de $\mathcal{L}$:

Label de couche	node_type (exemples)	Origine système	Exemple de node_id
physical	Core_Router, Edge_Router, OLT, FiberSpan	OSS / NID	PHY-OLT-PARIS-01
logical	VLAN, BGP_Session, MPLS_LSP	OSS / CMDB	LOG-BGP-PAR01-LYN02
service	VPN_Enterprise, Internet_Access, VoIP	BSS / Provisioning	SRV-VPN-12345
business	Customer, Contract, SLA_Tier	BSS / CRM	BIZ-CUST-998877

TABLE 4 – Instanciation du nœud TelecomNode par couche fonctionnelle

2.2.2 Les relations : un vocabulaire minimal et directionnel

Le choix d'un vocabulaire de relations restreint est tout aussi déterminant que celui du schéma de nœuds. Dans cette ontologie, l'ensemble des relations est réduit à quatre types directionnels, chacun encodant une traversée inter-couche précise :

$$\mathcal{R} = \{\text{CONNECTS_TO}, \text{RUNS_ON}, \text{DEPENDS_ON}, \text{SUBSCRIBED_TO}\} \quad (1)$$

Ces quatre relations définissent la *direction causale* du graphe, de la couche physique vers la couche métier :

Nœud source	Nœud cible	Relation	Sémantique
physical	physical	CONNECTS_TO	Lien physique bidirectionnel entre équipements (câble optique, lien Ethernet)
logical	physical	RUNS_ON	Une entité logique (protocole, tunnel) s'exécute sur un substrat physique
service	logical	DEPENDS_ON	Un service commercial dépend d'un ou plusieurs éléments logiques
business	service	SUBSCRIBED_TO	Un client ou un contrat est rattaché à un service actif

TABLE 5 – Vocabulaire relationnel de l'ontologie TelecomNode

La direction des relations est sémantiquement chargée : elle encode la *dépendance causale* ascendante, du substratum matériel vers l'impact contractuel. Une panne se propage dans le sens opposé à la direction des relations, ce qui permet de formuler l'analyse d'impact comme une traversée de graphe orienté en sens inverse (*reverse traversal*).

2.2.3 Justification mathématique de l'aplatissement ontologique

Soit un graphe de connaissances $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ où $\mathcal{V}$ représente l'ensemble des nœuds et $\mathcal{E}$ l'ensemble des arêtes directionnelles. Dans une ontologie télécom non optimisée, le nombre d'étiquettes distinctes de nœuds $|\mathcal{L}_{\mathcal{V}}|$ et de types de relations $|\mathcal{L}_{\mathcal{E}}|$ peut excéder plusieurs dizaines. Lors de la génération d'une requête Cypher pour une traversée à k sauts, la complexité de l'espace de recherche syntaxique soumis au LLM croît de manière exponentielle, proportionnellement à :

$$O(|\mathcal{L}_{\mathcal{V}}|^k \times |\mathcal{L}_{\mathcal{E}}|^k)$$

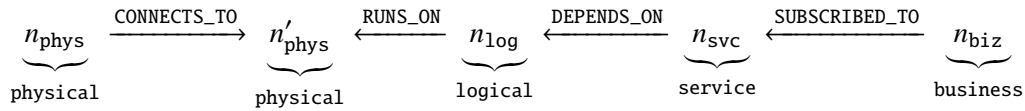
Cette complexité dimensionnelle sature inutilement la fenêtre de contexte du modèle en tokens de description schématique, et dégrade la précision du routage sémantique lors de la génération de requêtes structurées. En contraignant l'ontologie à une structure aplatie reposant sur une étiquette de base unique (TelecomNode) et quatre sous-couches canoniques $\mathcal{L}_{\mathcal{V}} = \{\text{physical}, \text{logical}, \text{service}, \text{business}\}$, on réduit statiquement $|\mathcal{L}_{\mathcal{V}}|$ à 4 et $|\mathcal{L}_{\mathcal{E}}|$ à 4. L'espace d'états de la requête devient formellement borné, avec une complexité réduite à :

$$O(4^k)$$

L'aplatissement agit dès lors comme une fonction de régularisation structurelle. Il minimise le coût calculatoire du traitement du contexte tout en maximisant la précision du ciblage topologique, garantissant une convergence déterministe lors de la génération du code d'inférence.

2.2.4 Synthèse schématique de la pile ontologique

L'articulation des quatre couches et de leurs quatre relations directionnelles forme une chaîne causale linéaire, représentée formellement comme suit :



Cette structure linéaire implique qu'une défaillance sur un nœud de couche **physical** se propage vers la couche **business** par traversée inverse de cette chaîne, propriété exploitée par le moteur GraphRAG au chapitre 3.

2.2.5 Étude de cas : requête d'impact en langage Cypher

Un nœud physique de type OLT portant l'identifiant **PHY-OLT-PARIS-13-01** passe à l'état "Down" à la suite d'une interruption de fibre. L'objectif est d'identifier automatiquement l'ensemble des clients impactés, ainsi que leur niveau de SLA contractuel, afin de prioriser les communications et d'évaluer le risque de pénalité financière.

La traversée s'effectue en sens inverse de la direction causale des relations, depuis le nœud physique en défaut jusqu'aux entités de couche **business** :

```
// Identifier tous les clients impactés par la défaillance d'un nœud physique,
// en traversant la pile ontologique de la couche physique vers la couche business.
```

```
MATCH (failed_node:TelecomNode:physical {node_id: 'PHY-OLT-PARIS-13-01'})
```

```
// Couche logique : entités s'exécutant sur le nœud défaillant
<-[:RUNS_ON]-(log_node:TelecomNode:logical)
```

```
// Couche service : services dépendant des entités logiques impactées
<-[:DEPENDS_ON]-(svc_node:TelecomNode:service)
```

```
// Couche business : clients ayant souscrit aux services impactés
<-[:SUBSCRIBED_TO]-(biz_node:TelecomNode:business)
```

```
WHERE biz_node.node_type = 'Customer'
```

```
RETURN
```

```
    biz_node.node_id      AS customer_id,
    biz_node.name         AS customer_name,
    biz_node.sla          AS sla_target_pct,
    svc_node.node_id      AS impacted_service_id,
    svc_node.name         AS impacted_service_name,
```

```
log_node.node_id      AS failed_logical_entity
```

```
ORDER BY biz_node.sla DESC
```

La requête traverse en quatre sauts la totalité de la pile ontologique (de la couche `physical` à la couche `business`) en exploitant exclusivement les quatre types de relations définis dans $\mathcal{R}$. Le tri descendant sur `sla` permet d'identifier en priorité les clients soumis aux engagements de disponibilité les plus contraignants.

2.3 Modélisation Orientée IA : Du Stockage à l'Inférence

L'ontologie présentée en section 2.2 ne vise pas uniquement à stocker l'information efficacement, mais à la rendre *navigable et inférable par un modèle génératif*. Un graphe de connaissances peut être conçu selon deux philosophies : organiser des données de manière exhaustive, à la façon d'une base de données documentaire, ou les structurer de façon à ce qu'un LLM puisse les parcourir et générer des requêtes Cypher valides avec un minimum d'ambiguïté résiduelle. Ces deux objectifs ne conduisent pas aux mêmes choix de modélisation. Trois principes ont guidé les décisions architecturales à cet égard.

2.3.1 Désambiguïssation par frontières de couches et typage de propriétés

Une approche naïve de la modélisation ontologique consisterait à définir un label de nœud distinct pour chaque type d'entité du domaine télécom : `OLT`, `CoreRouter`, `EdgeRouter`, `BGPSession`, `MPLSTunnel`, `VoIPService`, `Contract`, etc. Cette stratégie, dite à *expressivité maximale par prolifération de labels*, produit en apparence un graphe plus lisible pour un expert humain. Pour un LLM, elle constitue une source d'échec systématique.

En effet, un modèle génératif résout l'ambiguïté non pas par lecture de documentation externe, mais par *contrainte structurelle imposée dans son contexte*. Or, un schéma à $|\mathcal{L}_{\mathcal{V}}| \approx 20$ labels force le LLM à maintenir simultanément en attention l'ensemble de ces catégories lors de la génération de chaque clause `MATCH`. La probabilité qu'il invente un label syntaxiquement plausible mais absent du graphe (phénomène qualifié d'*hallucination schématique*) croît proportionnellement à $|\mathcal{L}_{\mathcal{V}}|$ [5].

L'ontologie retenue résout ce problème par une stratégie radicalement différente : la *désambiguïssation par frontières de couches et typage de propriétés*. Elle repose sur deux mécanismes complémentaires :

1. *Frontières de couches canoniques* : l'espace de navigation est partitionné en quatre domaines sémantiques disjoints et exhaustifs ($\mathcal{L}_{\mathcal{V}} = \{\text{physical}, \text{logical}, \text{service}, \text{business}\}$). Le LLM n'a jamais à choisir parmi des dizaines de labels : à chaque saut de traversée, il sélectionne l'une de ces quatre couches, dont la sémantique est univoque et stable. Cette partition réduit l'espace de décision par clause de $|\mathcal{L}_{\mathcal{V}}|$ à 4, soit un facteur de réduction de $\frac{|\mathcal{L}_{\mathcal{V}}|}{4}$ sur chaque étape de génération.
2. *Typage intra-couche par propriété `node_type`* : la spécialisation sémantique fine (distinguer un `Core_Router` d'un `Edge_Router` au sein de la couche `physical`, ou un `Customer` d'un `Contract` au sein de la couche `business`) est déléguée à un filtre sur la propriété scalaire `node_type`. Un filtre de propriété s'exprime en Cypher par une clause `WHERE` ou un prédicat inline, dont la syntaxe est invariante quel que soit le type d'entité. Le LLM n'a donc jamais à mémoriser une taxonomie de labels : il applique toujours le même patron syntaxique.

La comparaison formelle entre les deux approches illustre l'avantage de cette stratégie. Pour localiser un routeur de cœur de réseau, une ontologie à prolifération de labels produit :

```
MATCH (n:CoreRouter) RETURN n
```

L'ontologie condensée retenue produit :

```
MATCH (n:TelecomNode:physical {node_type: 'Core_Router'}) RETURN n
```

La seconde forme est syntaxiquement légèrement plus longue, mais elle est *structurellement contrainte* : le LLM ne peut y insérer qu'un label de couche appartenant à $\mathcal{L}_V$ et une valeur de chaîne pour `node_type`. Il ne peut pas inventer un label `CoreSwitch` ou `OpticalAmplifier` absent du schéma. La spécification de l'entité cible est ainsi découplée de la grammaire de labels (domaine où les hallucinations sont les plus fréquentes) pour être reportée dans un domaine de valeurs de chaînes, nettement plus robuste à la génération stochastique.

Cette architecture de désambiguïsation garantit que le LLM opère toujours sur un espace de labels de cardinalité $|\mathcal{L}_V| = 4$, quelle que soit la richesse sémantique effective du réseau modélisé.

2.3.2 L'optimisation de la fenêtre de contexte par récupération ciblée

La hiérarchie explicite `physical` $\rightarrow$ `logical` $\rightarrow$ `service` $\rightarrow$ `business`, matérialisée par les quatre relations directionnelles de $\mathcal{R}$, est précisément ce qui permet au moteur GraphRAG d'effectuer une *récupération topologique ciblée* plutôt que de charger l'intégralité du graphe en mémoire.

Un LLM dispose d'une fenêtre de contexte bornée [7, 8]. Fournir un export sérialisé de l'ensemble du réseau (comportant potentiellement plusieurs dizaines de milliers de nœuds `TelecomNode`) saturerait cette capacité et produirait une dégradation sévère des performances de raisonnement, phénomène empiriquement documenté sous le terme de *lost-in-the-middle* [8]. La structure en graphe hiérarchique permet au contraire une stratégie de *récupération par voisinage à k sauts (k-hop retrieval)* :

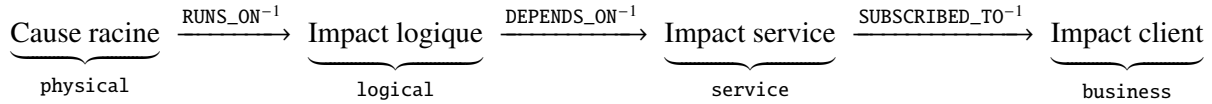
1. Le moteur identifie le nœud source de la défaillance dans la couche `physical` (ex : le nœud `TelecomNode:physical` dont `status` = "Down").
2. Seuls les nœuds voisins atteignables par traversée ascendante des relations $\mathcal{R}$ sont chargés en contexte : les nœuds `logical` liés par `RUNS_ON`, puis les nœuds `service` liés par `DEPENDS_ON`, puis les nœuds `business` liés par `SUBSCRIBED_TO`.
3. L'ensemble du graphe non concerné par la défaillance (représentant la grande majorité des nœuds) est ignoré sans coût supplémentaire.

La structure hiérarchique de l'ontologie est donc simultanément une contrainte de modélisation et une *fonction de compression sémantique* : elle délimite algorithmiquement le sous-graphe pertinent pour chaque requête d'analyse d'impact, sans nécessiter de mécanisme de filtrage externe.

2.3.3 Traçabilité et explicabilité par alignement causal

Le troisième principe concerne l'explicabilité du raisonnement produit par le système. Le modèle de données est structuré pour supporter nativement le raisonnement par *chaîne de causalité* (*Chain-of-Thought*) [19], technique qui améliore significativement la fiabilité des LLMs sur des tâches de diagnostic multi-étapes.

La direction des relations dans $\mathcal{R}$ reflète directement la structure de propagation d'une défaillance réseau :



En alignant le modèle de données sur cette chaîne causale, le graphe permet au LLM de générer des explications décomposées en étapes discrètes, chacune ancrée dans une arête vérifiable. À titre illustratif, la chaîne inférentielle complète pour un incident de type coupure de segment de fibre s'articule comme suit :

« Le nœud `TelecomNode:physical` de `node_type "FiberSpan"` portant l'identifiant `PHY-FIBER-PAR-13-042` est passé à l'état `Down` [*cause racine*]. En conséquence, les nœuds `TelecomNode:logical` liés par la relation `RUNS_ON` sur ce substrat physique (notamment la session `node_type = "BGP_Session" LOG-BGP-PAR01-LYN02`) sont indisponibles [*impact logique*]. Par propagation, le nœud `TelecomNode:service` `SRV-VPN-12345`, lié par `DEPENDS_ON` à cette entité logique, est interrompu [*impact service*]. Enfin, le nœud `TelecomNode:business` de `node_type "Customer"` `BIZ-CUST-998877`, lié par `SUBSCRIBED_TO` à ce service, est contractuellement impacté avec un risque de dépassement du seuil `sla = 99.9` [*impact client*]. »

Le graphe agit ici comme une *contrainte structurelle sur le raisonnement* : la direction des relations interdit au LLM d'établir un lien causal direct entre un nœud `physical` et un nœud `business` sans traverser les couches intermédiaires `logical` et `service`. Chaque affirmation du diagnostic est ainsi ancrée dans une séquence d'arêtes vérifiables, dont la validité peut être attestée par exécution de la requête Cypher correspondante sur l'état courant du graphe.

Synthèse du chapitre 2

Ce chapitre a posé les fondations nécessaires à l'architecture GraphRAG développée dans la suite du mémoire.

La *section 2.1* a diagnostiqué le problème structurel des opérateurs télécom : des données abondantes, mais cloisonnées dans des systèmes OSS et BSS qui ne partagent ni identifiants communs, ni mécanisme d'intégration temps réel. Les approches traditionnelles (entrepôts de données, bus d'intégration, MDM) se sont révélées insuffisantes, faute de traiter l'absence de *relations explicites* entre entités.

La *section 2.2* a défini l'ontologie qui répond à ce manque : un graphe de connaissances organisé en quatre couches canoniques (`physical`, `logical`, `service`, `business`), reliées par quatre relations directionnelles strictes (`CONNECTS_TO`, `RUNS_ON`, `DEPENDS_ON`, `SUBSCRIBED_TO`). La justification mathématique a établi que cette condensation ontologique réduit la complexité de l'espace de recherche syntaxique du LLM de $O(|\mathcal{L}_{\mathcal{V}}|^k \times |\mathcal{L}_{\mathcal{E}}|^k)$ à $O(4^k)$.

La *section 2.3* a justifié les trois principes de modélisation orientée IA : désambiguïsation par frontières de couches et typage de propriétés pour éliminer l'hallucination schématique, récupération topologique ciblée par traversée k -hop pour préserver la fenêtre de contexte, et alignement causal pour contraindre structurellement le raisonnement en chaîne du LLM.

Conçu comme un *espace sémantique navigable à cardinalité bornée*, ce graphe constitue la fondation sur laquelle repose l'intégralité de l'architecture GraphRAG. Le chapitre 3 détaillera comment cette structure est exploitée dynamiquement pour générer des diagnostics d'impact en temps réel à partir de requêtes formulées en langage naturel.

3 Architecture Système et Implémentation du GraphRAG

À retenir : Le rôle de chaque composant

Le système repose sur une architecture en trois parties :

1. *Moteur d'inférence sémantique (LLM)* : il comprend la question en langage naturel (français).
2. *Graphe de connaissances topologique (Neo4j)* : il contient la représentation exacte de tous les câbles et clients.
3. *Agent d'orchestration et de rétroaction (LangChain)* : il fait le lien entre la question et la carte topologique.

3.1 Pipeline de Données : De l'Extraction au Graphe de Connaissances

Objectif : Formaliser l'architecture industrielle cible du pipeline ETL destiné à alimenter le jumeau numérique, puis établir la justification épistémologique du recours à un environnement synthétique pour la validation expérimentale de ce mémoire.

3.1.1 Introduction : qualité des données et validité du raisonnement automatisé

Le chapitre 2 a formalisé l'ontologie du graphe de connaissances (son *schéma architectural* : l'entité centrale à modéliser (TelecomNode déclinée selon quatre labels de couche : physical, logical, service, business), les quatre relations directionnelles qui les articulent (CONNECTS_TO, RUNS_ON, DEPENDS_ON, SUBSCRIBED_TO), et les raisons structurelles pour lesquelles cette représentation aplatie est interprétable par un LLM sans ambiguïté de schéma). Il convient désormais de répondre à la question opérationnelle suivante : *par quel processus les données du monde réel (distribuées dans des systèmes hétérogènes, sémantiquement disjoints et soumis à des cycles de mise à jour propres) sont-elles transformées en ce graphe structuré, cohérent et interrogeable ?*

Ce rôle est dévolu au *pipeline ETL*⁴, qui constitue la colonne vertébrale de l'architecture GraphRAG. En son absence, le jumeau numérique se réduirait à un graphe formellement correct mais sémantiquement vide : les

4. ETL : *Extract, Transform, Load*. Paradigme standard en ingénierie des données désignant la séquence d'opérations consistant à extraire des données depuis des sources multiples, à les transformer pour en assurer l'homogénéité sémantique, puis à les charger dans un système cible.

nœuds existeraient sans relations topologiques fiables, ou pire, des arêtes incorrectes induiraient des diagnostics erronés sur l'état du réseau.

Ce pipeline se décompose en trois étapes séquentielles et complémentaires : *l'extraction* (collecte des données brutes depuis chaque système source), *la transformation* (normalisation et réconciliation sémantique des formats hétérogènes), et *le chargement* (matérialisation incrémentale du jumeau numérique dans Neo4j).

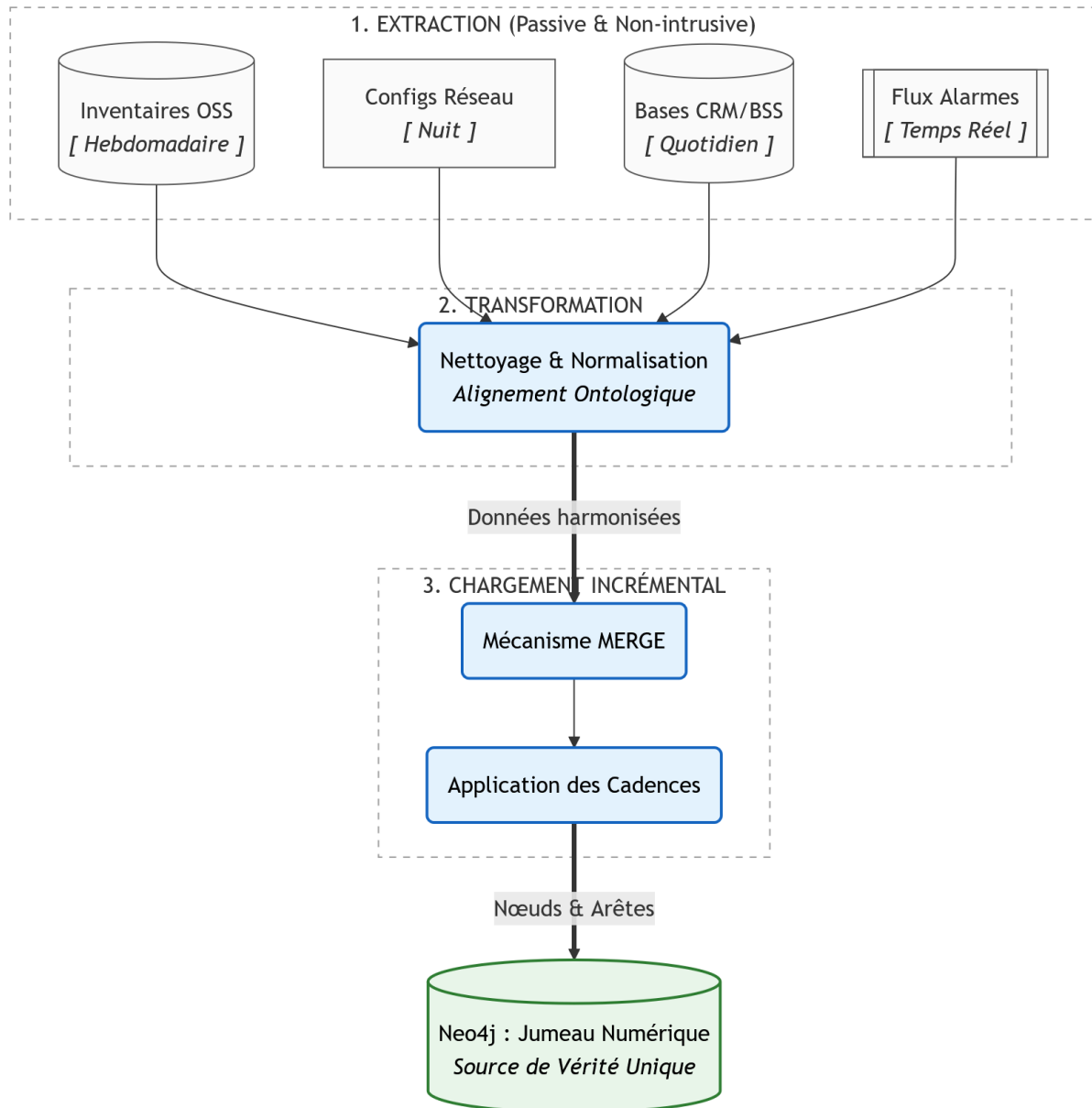


FIGURE 6 – Architecture fonctionnelle et flux de données du pipeline ETL vers le jumeau numérique

3.1.1.1 Note liminaire : architecture industrielle cible et contraintes de recherche académique L'architecture décrite dans cette section constitue l'*architecture industrielle cible* : elle représente le pipeline tel qu'il serait déployé en environnement opérateur réel, intégrant des flux en provenance de systèmes OSS/BSS actifs (Oracle, ServiceNow, NMS propriétaires, exports SNMP en temps réel).

Il est impératif de souligner que l'accès à ces systèmes de production est *strictement impossible dans le cadre d'une recherche académique*, pour des raisons cumulatives relevant de trois registres distincts :

1. *Confidentialité industrielle et accords de non-divulgence (NDA)* : les bases d'inventaire réseau (OSS), les référentiels clients (BSS/CRM) et les journaux d'incidents constituent des actifs stratégiques dont la divulgation est contractuellement prohibée.
2. *Contraintes réglementaires (RGPD)* : les données de provisioning et les contrats de service associés à des personnes physiques sont soumis au Règlement Général sur la Protection des Données (UE 2016/679), rendant leur extraction et leur stockage hors du périmètre de l'opérateur juridiquement irrecevable sans protocole de pseudonymisation certifié.
3. *Intégrité des systèmes de production* : toute connexion, même en lecture seule, à une base de données opérationnelle introduit un risque résiduel pour la disponibilité des services, incompatible avec les exigences de SLA des opérateurs télécoms.

En conséquence, la validation expérimentale de ce mémoire ne repose pas sur des données de production réelles.

3.1.1.2 Substitution méthodologique : le jumeau numérique synthétique Pour pallier ces contraintes sans sacrifier la rigueur scientifique de la démarche, le pipeline ETL industriel décrit ci-après est substitué par un *jumeau numérique synthétique (Synthetic Digital Twin)*, généré de manière procédurale et entièrement déterministe. Ce dispositif, dont la conception est détaillée à la section 3.3, constitue un environnement *sandbox* reproduisant fidèlement :

- les *structures de données* produites par le pipeline industriel (en particulier le schéma de nœuds `TelecomNode` avec ses quatre labels de couche, ses propriétés typées (`node_type`, `status`, `sla`) et ses identifiants normalisés), ainsi que les quatre relations directionnelles (`CONNECTS_TO`, `RUNS_ON`, `DEPENDS_ON`, `SUBSCRIBED_TO`) définies au chapitre 2 ;
- la *stratification ontologique à quatre couches* (`physical`, `logical`, `service`, `business`) avec les contraintes de directionnalité strictement contrôlées entre couches ;
- les *formats de fichiers intermédiaires* (CSV pour les nœuds, JSON pour les relations) que le pipeline ETL réel produirait comme artefacts de transformation avant chargement dans Neo4j.

Cette approche présente un avantage épistémologique décisif : elle garantit la *reproductibilité scientifique* des expérimentations. L'ensemble des scénarios de test (topologies réseau, incidents simulés, dépendances de services, contrats clients) est généré à partir d'une graine aléatoire fixée (*random seed*), permettant à tout chercheur tiers de régénérer exactement le même graphe et de reproduire les évaluations présentées au chapitre 4. Cette propriété est absente par définition de tout jeu de données de production.

La description qui suit présente donc cette *architecture industrielle cible* à titre de référence de conception, avant que la section 3.3 ne formalise la correspondance structurelle entre le pipeline réel et son substitut synthétique.

3.1.2 Étape 1 : l'extraction (collecte multi-sources et réconciliation d'états)

La première étape du pipeline consiste à collecter la matière première depuis l'ensemble des systèmes sources identifiés au chapitre 2. Cette collecte soulève une difficulté structurelle fondamentale : chaque système expose ses données dans un format, à une granularité et selon une sémantique qui lui sont propres, sans qu'aucun mécanisme natif de cohérence inter-systèmes ne soit garanti.

3.1.2.1 Hétérogénéité des sources et problème de réconciliation d'états Quatre familles de sources alimentent le graphe, chacune contribuant à une ou plusieurs couches de l'ontologie. Toutes convergent vers la même structure cible : le nœud `TelecomNode` et ses quatre relations directionnelles.

Les inventaires techniques (OSS) fournissent la liste exhaustive des équipements physiques (routeurs, OLT, interfaces, liaisons fibre) ainsi que leurs attributs (localisation géographique, capacité nominale, état opérationnel). Ces données se présentent typiquement sous forme de fichiers tabulaires (exports CSV ou Excel) extraits périodiquement depuis des NMS⁵ propriétaires. Elles alimentent principalement la couche `physical` de l'ontologie, peuplant les nœuds `TelecomNode` dont la propriété `node_type` prend des valeurs telles que `"Core_Router"`, `"OLT"` ou `"Fiber_Link"`.

Les fichiers de configuration réseau contiennent la description précise des interconnexions logiques entre équipements : assignation des interfaces, activation des protocoles de routage, tables de commutation. Ces fichiers, générés directement par les équipements (Cisco IOS, Juniper JunOS) au format texte structuré, nécessitent une phase d'*analyse syntaxique (parsing)* pour en extraire les relations sous-jacentes. Une fois analysées, ces interconnexions se matérialisent dans le graphe exclusivement sous la forme de relations `CONNECTS_TO` (liens physiques entre nœuds de couche `physical`) et `RUNS_ON` (sessions logiques s'appuyant sur un substrat physique, entre couches `physical` et `logical`).

Les bases CRM et de provisioning (BSS) fournissent les données commerciales et contractuelles : identifiants clients, contrats de service, SLA, offres souscrites. Ces données résident dans des bases relationnelles transactionnelles (Oracle, SAP) et sont extraites via des requêtes SQL normalisées. Elles alimentent les couches `service` et `business`, articulées dans le graphe par les relations `DEPENDS_ON` (un service dépend de ressources logiques) et `SUBSCRIBED_TO` (un client de couche `business` est abonné à un service de couche `service`).

Les flux d'événements réseau constituent la dimension temporelle du pipeline : traps SNMP⁶, journaux d'incidents et tickets issus des plateformes ITSM (ServiceNow). Ces flux ne décrivent pas l'état statique du réseau, mais ses transitions d'état (les dégradations, les ruptures de liens, les interventions correctives). Ils actualisent dynamiquement la propriété `status` des nœuds `TelecomNode` existants sans nécessiter la création de nouvelles entités ni l'introduction de nouveaux types de relations.

La difficulté centrale de cette étape réside dans le *problème de réconciliation d'états multi-sources (multi-source state reconciliation)* : pour une même entité physique ou logique, plusieurs systèmes peuvent simultanément décrire des états contradictoires ou sémantiquement non alignés. Ainsi, un équipement en panne peut être représenté comme `status = "DOWN"` dans le NMS, `etat = 0` dans la base d'inventaire, et `operational_state = "inactive"` dans le système de ticketing. La résolution de ces conflits de représentation exige la définition d'une *politique de priorité inter-sources* : pour chaque attribut d'un nœud `TelecomNode`, le pipeline doit définir quelle source fait autorité en cas de discordance.

3.1.2.2 Principe de non-intrusivité Un invariant architectural guide cette étape : le pipeline opère exclusivement en *lecture passive* sur les systèmes sources. Aucune écriture, aucune modification, aucune transaction n'est initiée sur les bases de production. Cette contrainte est à la fois technique (préservation de l'intégrité tran-

5. NMS : *Network Management System*. Outil logiciel permettant la supervision et la gestion centralisée des équipements d'un réseau de télécommunications.

6. SNMP : *Simple Network Management Protocol*. Protocole permettant aux équipements réseau de remonter automatiquement des notifications d'état (« traps ») vers les systèmes de supervision lors de la survenue d'un événement.

sactionnelle des systèmes sources) et organisationnelle (condition nécessaire à l'adhésion des équipes métier et à la conformité avec les politiques de sécurité des opérateurs). Le jumeau numérique est un *reflet* du réseau opérationnel, non un système de contrôle.

3.1.3 Étape 2 : la transformation (normalisation sémantique et alignement ontologique)

L'étape de transformation constitue le cœur intellectuel du pipeline. La collecte de données brutes ne suffit pas : il faut résoudre le problème de réconciliation d'états multi-sources identifié à l'étape précédente, et ramener l'ensemble des représentations hétérogènes à un vocabulaire unique, directement aligné sur l'ontologie aplatie définie au chapitre 2.

Cette normalisation s'opère en deux temps. Dans un premier temps, les valeurs d'attributs sont *standardisées* par application d'un ensemble de règles de correspondance (*mapping rules*) : toutes les variantes sémantiques décrivant un même état opérationnel sont ramenées à une valeur canonique unique du champ *status*. Dans un second temps, les entités extraites sont *alignées* sur le schéma ontologique : chaque objet source est instancié comme un nœud *TelecomNode* portant le label de couche approprié parmi *physical*, *logical*, *service* ou *business*, et ses attributs sont validés contre le schéma de propriétés attendu (*node_type*, *status*, *sla*).

Cette étape produit en sortie des fichiers intermédiaires structurés (CSV pour les nœuds, JSON pour les relations) qui constituent l'interface contractuelle entre le pipeline ETL et le module de chargement. C'est précisément ce format intermédiaire (nœuds *TelecomNode* typés et relations issues du vocabulaire strict à quatre éléments) que reproduit le jumeau numérique synthétique décrit à la section 3.3.

3.1.4 Étape 3 : le chargement (construction incrémentale du jumeau numérique dans Neo4j)

La troisième étape matérialise dans Neo4j l'ensemble des nœuds et des arêtes produits par les étapes précédentes. C'est le moment où le schéma ontologique prend corps sous la forme d'un graphe de connaissances réel et interrogeable par l'architecture GraphRAG.

Une erreur de conception courante consisterait à vider et reconstruire intégralement le graphe à chaque exécution du pipeline. Cette approche est à la fois coûteuse en ressources de calcul et incompatible avec les contraintes de disponibilité d'un système de supervision opérationnel. Le pipeline adopte en conséquence une stratégie *incrémentale* : seules les entités nouvelles ou modifiées depuis la dernière exécution sont intégrées.

Cette logique repose sur la sémantique de l'opérateur *MERGE* de Cypher : avant de créer un nœud ou une arête, le système vérifie d'abord son existence par correspondance sur la propriété identifiante. En cas de correspondance, les propriétés sont mises à jour ; en l'absence de correspondance, l'entité est créée. Ce mécanisme garantit l'*idempotence* du pipeline : des exécutions répétées à partir des mêmes données d'entrée produisent un état final identique du graphe, sans accumulation de doublons ni de contradictions.

Les cadences d'exécution sont différenciées selon la volatilité propre à chaque source de données :

Source	Cadence de mise à jour	Justification
Alarmes réseau (SNMP)	Temps réel (< 30 s)	Données critiques pour la détection d'incidents en cours d'occurrence
Configurations d'équipements	Quotidienne (nuit)	Évolution lente ; extraction en période creuse pour minimiser l'impact
Inventaire réseau (OSS)	Hebdomadaire	Données structurellement stables (adjonctions d'équipements peu fréquentes)
Données CRM / Contrats (BSS)	Quotidienne	Fréquence élevée de commandes, modifications et résiliations

TABLE 6 – Cadences de mise à jour différenciées du pipeline ETL selon la volatilité des sources

Cette stratégie à cadences différenciées garantit que la couche la plus critique (l'état temps réel des nœuds TelecomNode de couche physical) reste synchronisée avec le réseau opérationnel, sans induire de charge superflue sur les systèmes sources pour des données à évolution lente.

3.1.5 Synthèse : un pipeline au service d'une source de vérité unique

Le pipeline ETL formalisé dans cette section constitue, en un sens, la composante la moins visible de l'architecture GraphRAG, mais sans doute la plus déterminante pour sa validité opérationnelle. Aucun modèle de raisonnement, aussi sophistiqué soit-il, ne peut compenser des données mal préparées, des relations manquantes ou des états contradictoires non résolus. Le principe classique de l'ingénierie des données (« *Garbage In, Garbage Out* ») s'applique avec une acuité particulière dans un contexte GraphRAG : une arête incorrecte dans le graphe ne dégrade pas seulement la précision d'une réponse, elle peut propager une inférence causale erronée à travers plusieurs niveaux de dépendances.

Ce pipeline remplit une mission fondamentale : garantir que le jumeau numérique constitue une *source de vérité unique* (*Single Source of Truth*) cohérente, à jour et exploitable par le LLM. C'est sur cette fondation que repose l'intégralité de la valeur de l'architecture GraphRAG. Lorsqu'un opérateur formule une requête en langage naturel (« *Quels abonnés sont affectés par l'interruption sur le lien Paris-Lyon ?* »), la fiabilité de la réponse est bornée par la qualité du graphe qui la sous-tend.

3.2 Orchestration de l'IA : Du Langage Naturel à la Requête de Graphe

La section précédente a établi que le jumeau numérique constitue une *source de vérité unique* dont la cohérence repose sur un pipeline ETL rigoureux. Cette fondation déterministe est nécessaire, mais non suffisante : elle reste muette face à une question formulée en français par un opérateur d'astreinte. Comblé ce fossé entre la fluidité du langage humain et la rigueur formelle d'une base de données de graphe constitue l'objet de la présente section.

3.2.1 Le problème fondamental : traduire l'intention en instruction

Pour interroger Neo4j, un ingénieur doit maîtriser le langage Cypher (un langage de requête formel, précis et non tolérant à l'ambiguïté). Une question comme « *Quels clients risquent une pénalité SLA si la fibre de Lyon n'est pas rétablie dans les deux heures ?* » se traduit en une séquence d'instructions techniques que seul un spécialiste peut formuler. Exiger cette compétence de chaque opérateur, manager ou ingénieur d'astreinte est irréaliste.

La solution intuitive consiste à connecter un LLM directement à Neo4j et à lui transmettre l'intégralité du schéma du graphe afin qu'il génère lui-même les requêtes Cypher appropriées. Cette approche, dite d'*injection de schéma statique*, constitue le point de départ naturel. Ses limites structurelles, examinées ci-après, motivent l'architecture plus robuste proposée dans ce mémoire.

3.2.2 Les limites structurelles de l'injection de schéma statique

3.2.2.1 Saturation de la mémoire de travail contextuelle La fenêtre de contexte d'un LLM peut être assimilée à une *mémoire de travail* de capacité finie : tout ce que le modèle peut « voir » lors d'un raisonnement y est inscrit, et rien d'autre. Dans sa forme la plus naïve, l'injection de schéma statique consiste à transmettre au LLM une description textuelle exhaustive de l'ensemble des entités et des relations du graphe, sans sélection ni filtrage. Un schéma non optimisé (incluant des dizaines de types de nœuds hétérogènes, autant de variantes de relations, et l'intégralité de leurs propriétés) génère aisément un contexte de plusieurs milliers de tokens. Bien que la fenêtre de contexte soit en constante expansion⁷, l'injection d'un schéma verbeux contraint mécaniquement l'espace disponible pour la question elle-même, les exemples de requêtes et l'historique conversationnel.

3.2.2.2 Dilution de l'attention et hallucination de schéma Le problème n'est pas uniquement quantitatif. Le mécanisme d'attention des Transformers distribue son budget d'attention sur l'intégralité du contexte disponible. Un schéma verbeux noie mécaniquement les signaux pertinents : le modèle accorde autant d'attention aux éléments de schéma sans rapport avec la question posée qu'aux entités directement concernées. Cette dilution conduit à un phénomène documenté dans la littérature : l'*hallucination de schéma* [21], dans laquelle le modèle génère des requêtes syntaxiquement plausibles mais sémantiquement incorrectes (réfrençant des noms de relations inexistantes ou des propriétés fantaisistes).

3.2.2.3 Absence de contextualisation pragmatique L'injection de schéma statique souffre d'une troisième lacune : elle transmet la *grammaire* du graphe, mais non son *usage*. Un schéma indique qu'il existe une relation SUBSCRIBED_TO entre les nœuds TelecomNode de couche business et de couche service, mais ne montre pas comment cette relation doit être traversée pour répondre à une question de type « calcul de risque de pénalité ». Ce fossé entre connaissance structurelle et connaissance pragmatique est comblé, en ingénierie des LLMs, par le *few-shot prompting* (la fourniture d'exemples concrets et validés [22]). L'architecture statique, par définition, ne peut pas fournir ces exemples de manière dynamique et contextualisée.

7. À titre indicatif, GPT-4o dispose d'une fenêtre de 128 000 tokens et Claude 3.5 Sonnet d'une fenêtre de 200 000 tokens. Les études empiriques montrent cependant que les performances de récupération d'information se dégradent significativement pour les éléments situés en milieu de fenêtre (phénomène connu sous le nom de *lost in the middle* [20]).

Ces trois limitations (saturation contextuelle, dilution de l'attention, absence d'exemples pertinents) convergent vers le même diagnostic : l'injection de schéma statique non contrôlé n'est pas une architecture viable pour un déploiement industriel. Elles valident a posteriori le choix d'une ontologie aplatie et motivent une approche d'orchestration fondamentalement différente, fondée sur la sélection dynamique de contexte.

3.2.3 Context engineering et routage sémantique dynamique

3.2.3.1 Du schéma statique au contexte ciblé La réponse architecturale aux limites identifiées repose sur un principe fondateur du *context engineering*⁸ : *ne fournir au LLM que le contexte strictement nécessaire pour répondre à la question posée*. Appliqué au problème étudié, ce principe se traduit par une architecture hybride en deux couches : un *routage sémantique vectoriel* pour la sélection contextuelle, et une *traversée déterministe du graphe* pour l'exécution. Cette architecture est rendue possible par la compacité de l'ontologie aplatie retenue : le schéma partiel injecté dynamiquement dans le prompt est toujours un sous-ensemble d'un vocabulaire déjà minimal.

3.2.3.2 La bibliothèque de requêtes validées : une mémoire institutionnelle du système La première composante de cette architecture est une *bibliothèque de requêtes Cypher validées* (*Cypher Query Library*), constituée progressivement à partir de deux sources : les requêtes générées par le système et validées par les opérateurs lors d'incidents réels, et un corpus initial rédigé par des experts lors de la phase de déploiement. Chaque entrée de cette bibliothèque est formalisée comme un triplet structuré :

$$\langle q_{\text{nat}}, q_{\text{cypher}}, \mathcal{S}_{\text{partiel}} \rangle$$

où q_{nat} désigne la question en langage naturel, q_{cypher} la requête Cypher correspondante validée, et $\mathcal{S}_{\text{partiel}}$ le sous-ensemble du schéma ontologique impliqué par cette requête (par exemple, uniquement les nœuds *TelecomNode* de couche *service* et *business* reliés par *SUBSCRIBED_TO*, pour une question de type SLA, ou les couches *physical* et *logical* reliées par *CONNECTS_TO* et *RUNS_ON* pour une question de diagnostic topologique).

Chaque triplet est encodé sous forme d'un vecteur sémantique et indexé dans *ChromaDB*, une base de données vectorielle qui permet de retrouver, par similarité cosinus, les entrées dont la question en langage naturel est sémantiquement la plus proche de la question courante. Ce mécanisme constitue le *routage sémantique* : pour toute nouvelle question, le système identifie les k triplets les plus pertinents (avec $k = 3$ dans la configuration retenue) et extrait leurs composantes $q_{\text{cypher}}^{(i)}$ et $\mathcal{S}_{\text{partiel}}^{(i)}$ pour construire le contexte enrichi transmis au LLM.

3.2.3.3 Construction du prompt dynamique La construction du prompt (ou *system prompt*) finalement transmis au LLM ne repose sur aucun calcul complexe, mais sur une *concaténation structurée de chaînes de caractères*. Le framework d'assemblage génère dynamiquement ce texte en y injectant trois blocs d'information précis :

- *Le sous-schéma strict (schema pruning)* : plutôt que de fournir l'intégralité du modèle de la base, le système fusionne uniquement les nœuds et relations présents dans les k exemples sélectionnés. En

8. *Context Engineering* : discipline émergente visant à optimiser le contenu, la structure et la taille du contexte fourni à un LLM, en opposition à la pratique naïve consistant à injecter le maximum d'informations disponibles [23].

termes d'implémentation, cela se traduit par l'injection d'une courte liste textuelle restrictive directement dans le prompt (par exemple : Entités autorisées : TelecomNode[physical, logical] | Relations autorisées : CONNECTS_TO, RUNS_ON). Ce formatage explicite agit comme un garde-fou syntaxique.

- *Les k exemples d'apprentissage (few-shot)* : les paires de questions/requêtes $\langle q_{\text{nat}}^{(i)}, q_{\text{cypher}}^{(i)} \rangle$ issues du routage sémantique sont insérées littéralement dans le texte. Elles illustrent concrètement la trajectoire attendue. Le LLM lit par exemple : « Exemple 1 - Question : Quel est l'impact de la coupure fibre ? $\rightarrow$ Cypher : MATCH (n :TelecomNode {layer : 'physical'}) ... ».

Cette méthode d'assemblage éclaire la réalité de l'implémentation : elle ramène l'interrogation complexe d'une topologie réseau à un *exercice de complétion de texte sous contrainte*. Le LLM n'a pas besoin de modéliser mathématiquement le réseau télécom ; il lui suffit d'appliquer par mimétisme un motif syntaxique éprouvé, sur un vocabulaire explicitement restreint à quelques dizaines de *tokens*.

3.2.4 L'agent orchestrateur : de la chaîne séquentielle à la boucle de rétroaction

Le *context engineering* garantit la pertinence des instructions envoyées au LLM, mais il ne définit pas l'exécution de ces instructions. Relier la génération probabiliste du langage naturel à l'interrogation déterministe d'une base de graphe requiert une couche de coordination explicite, désignée sous le terme d'*orchestration*.

3.2.4.1 De LangChain à LangGraph : dépasser la linéarité Dans sa forme naïve, l'orchestration est souvent implémentée via des pipelines séquentiels (tels que le framework standard *LangChain* [24]), où les données transitent de manière unidirectionnelle : (1) extraction du contexte, (2) génération de la requête, (3) exécution, (4) synthèse. Si cette architecture est efficace pour le chemin nominal, elle est structurellement inadaptée aux environnements critiques : la moindre erreur de syntaxe Cypher générée par le LLM provoque un échec irrémédiable du pipeline, sans possibilité de correction.

Pour résoudre cette fragilité, l'architecture proposée abandonne le modèle linéaire au profit d'un *graphe orienté d'états* implémenté via *LangGraph* [25]. La différence architecturale fondamentale réside dans la *capacité à former des cycles* : la progression n'est plus automatique, mais soumise à des arêtes conditionnelles. Si une erreur survient, le contrôle peut être renvoyé à un état antérieur pour itérer sur le traitement.

3.2.4.2 Implémentation de la boucle corrective et gestion des erreurs Le pipeline d'orchestration est modélisé sous *LangGraph* comme un graphe de six nœuds fonctionnels : (1) le routage sémantique, (2) la génération Cypher, (3) la validation syntaxique, (4) l'exécution Neo4j, (5) la synthèse linguistique, et (6) la mise à jour de la mémoire conversationnelle.

La figure 7 illustre ce pipeline. Après que le LLM a généré une requête Cypher q_{cypher} , celle-ci est soumise à une validation syntaxique (nœud 3). Trois cas de figure conditionnent la traversée du graphe :

1. *Exécution réussie avec résultats* : le pipeline progresse vers l'étape de synthèse. Il s'agit du chemin nominal.
2. *Erreur syntaxique Cypher (hallucination de relation)* : Neo4j retourne une exception de parsing. Le LLM a généré une relation verbosée (ex : LINKED_TO) au lieu du vocabulaire strict de l'ontologie

(CONNECTS_T0). LangGraph capture cette exception de type `SyntaxException`, la formate sous la forme d'un message d'erreur structuré, et renvoie le contrôle au nœud de génération (nœud 2) en enrichissant le prompt avec l'erreur. Le LLM est explicitement contraint de corriger sa syntaxe.

3. *Résultat vide sémantiquement incohérent* : la requête s'exécute sans erreur syntaxique mais retourne un ensemble vide alors que la question implique un résultat. LangGraph détecte cette incohérence par un heuristique de validation et renvoie le contrôle au LLM avec un avertissement explicite.

Dans les cas 2 et 3, le nombre d'itérations de la boucle de rétroaction est borné à $N_{\max} = 3$

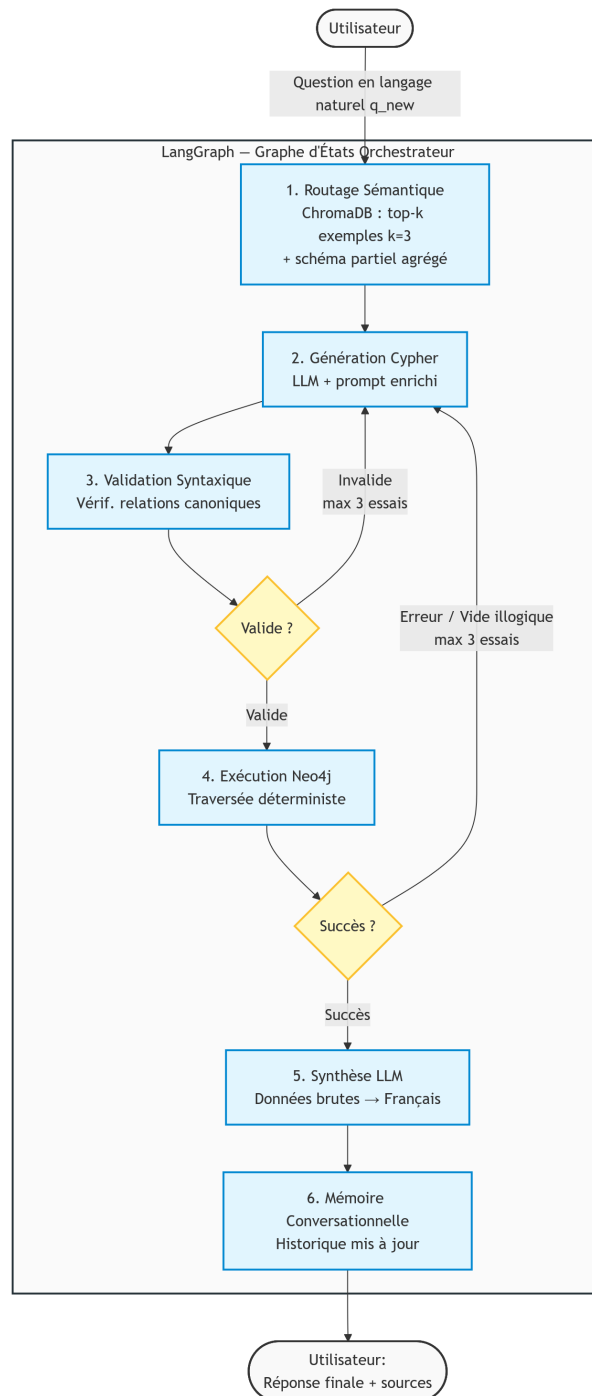


FIGURE 7 – Pipeline d'orchestration Text-to-Cypher-to-Answer avec routage sémantique dynamique et boucle de rétroaction corrective (LangGraph). Les nœuds (1) à (6) correspondent aux états du graphe LangGraph. L'étape (3) valide que les relations générées appartiennent exclusivement au vocabulaire canonique de l'ontologie aplatie. Les arêtes de rétroaction vers le nœud (2) matérialisent les cycles de correction, bornés à $N_{\max} = 3$ itérations.

3.2.5 Synthèse opérationnelle

Le résultat brut retourné par Neo4j (typiquement une liste de lignes et de colonnes) n'est pas directement exploitable par un opérateur sous pression. Une réponse du type :

```
[{customer.name: "Société DUPONT", sla.penalty_eur: 15000},  
 {customer.name: "Groupe MARTIN", sla.penalty_eur: 8500}]
```

est précise, mais peu actionnable dans l'urgence d'un incident. LangGraph transmet ces données brutes au LLM une seconde fois, accompagnées de la question originale, en lui demandant de produire une réponse en langage naturel, claire, contextualisée et directement opérationnelle. Le LLM joue alors son rôle naturel de *synthétiseur linguistique* [2] (reformuler des données factuelles en prose intelligible) :

« Suite à la coupure fibre détectée à 03h17, deux clients entreprise sont actuellement impactés avec un risque de pénalité SLA : la Société DUPONT (15 000 €) et le Groupe MARTIN (8 500 €). La restauration du service doit intervenir avant 07h17 pour éviter le déclenchement contractuel des pénalités. »

Cette réponse est *entièrement ancrée dans les données du graphe* : aucun chiffre, aucun nom, aucun délai n'a été inventé par le modèle.

Synthèse : une interface conversationnelle sur des données déterministes

Le pipeline d'orchestration décrit dans cette section constitue l'interface entre deux univers de nature radicalement différente : la génération probabiliste du langage humain et la rigueur formelle d'une base de données de graphe. La progression architecturale suivie (du LLM isolé au pipeline séquentiel LangChain, puis au graphe d'états cyclique LangGraph) répond à une série de contraintes identifiées séquentiellement : l'impossibilité pour un opérateur de formuler des requêtes Cypher, les limites de l'injection de schéma statique, et la fragilité inhérente aux pipelines sans rétroaction. En substituant à l'injection de schéma statique un *context engineering* dynamique fondé sur le routage sémantique, et en encapsulant l'exécution dans une boucle de rétroaction corrective à terminaison garantie, l'architecture atteint un niveau de robustesse compatible avec un déploiement industriel. Le tableau 7 résume la séparation stricte des responsabilités qui en découle.

Composant	Responsabilité	Ce qu'il ne fait pas
LLM	Comprendre la question, générer Cypher à partir du contexte enrichi, rédiger la réponse finale en langage naturel, s'auto-corriger sur retour d'erreur	Inventer des faits, accéder directement aux données, décider du flux d'exécution
ChromaDB	Sélectionner dynamiquement, par routage sémantique, les exemples few-shot et le schéma partiel pertinents pour la question posée	Exécuter des requêtes sur le graphe, interpréter le langage naturel
LangChain	Assembler séquentiellement les composants du pipeline et transmettre les sorties d'un composant comme entrées du suivant dans un format conforme	Gérer les boucles de rétroaction, maintenir un état conditionnel entre les étapes
LangGraph	Orchestrer le graphe d'états, valider la conformité des relations générées au vocabulaire canonique, gérer la boucle de rétroaction (max 3 itérations), maintenir la mémoire conversationnelle, garantir la terminaison formelle du pipeline	Interpréter le langage, raisonner sur la topologie réseau
Neo4j	Exécuter les traversées déterministes de graphe en $O(4^k)$, garantir l'exactitude et la traçabilité factuelle de chaque résultat retourné	Comprendre le langage naturel, inférer des intentions, tolérer des requêtes malformées

TABLE 7 – Séparation des responsabilités entre les cinq composants de l'architecture GraphRAG étendue. LangChain et LangGraph sont distingués pour refléter leur rôle fonctionnel respectif : assemblage séquentiel pour le premier, gestion des cycles conditionnels pour le second.

Cette séparation stricte est le fondement de la *fiabilité* du système. Elle garantit qu'aucun composant ne déborde sur le domaine d'un autre : le LLM ne statue jamais sur les faits, Neo4j ne tente jamais d'interpréter une intention humaine, et LangGraph n'analyse jamais le contenu sémantique (il orchestre uniquement les transitions d'états, en s'assurant notamment que les relations Cypher générées respectent le vocabulaire canonique de l'ontologie aplatie). C'est leur articulation précise qui produit, in fine, une intelligence opérationnelle réellement exploitable à l'échelle industrielle.

4 Évaluation et Benchmarking

Objectif : Soumettre l'architecture GraphRAG décrite au chapitre 3 à une évaluation quantitative rigoureuse, en la confrontant à un système RAG vectoriel standard constituant la ligne de base (baseline), afin d'établir empiriquement la supériorité structurelle de l'approche fondée sur la traversée déterministe de graphe pour les requêtes d'impact topologique dans les réseaux de télécommunication.

L'hypothèse centrale est la suivante : dès lors que la question posée implique une traversée structurelle de l'ontologie sur plusieurs sauts ($k \geq 2$), la recherche par similarité cosinus s'avère structurellement incapable de restituer l'exhaustivité du rayon d'impact (*blast radius*), engendrant ce que l'on nomme des *hallucinations structurelles* (à distinguer des hallucinations linguistiques classiques). Le protocole expérimental, les métriques d'évaluation et l'interprétation des résultats sont présentés dans les sections suivantes.

4.1 Protocole d'Expérimentation et Architecture de Test

4.1.1 Environnement d'exécution

L'intégralité de la campagne d'évaluation est pilotée par un script unique, `run_evaluation.py`, exécuté sur l'infrastructure d'inférence de *Groq*⁹, dont l'architecture matérielle repose sur des unités de traitement logique (*Language Processing Units*, LPU). Ce choix est motivé par la faible latence de décodage propre aux LPU, critique dans un pipeline de génération de requêtes Cypher multi-sauts.

Le protocole repose sur une architecture à double modèle (*Dual-Model*) instituant une séparation explicite entre le moteur de raisonnement et le juge d'évaluation. Le modèle *Llama-3.3-70b-Versatile* [26] assure la génération Cypher et la synthèse des réponses pour les deux pipelines, tandis que *GPT-4o* [27] joue le rôle d'*évaluateur impartial* (*LLM-as-a-judge*) pour le calcul des métriques RAGAS. Ce découplage est délibéré : il neutralise le biais d'auto-évaluation (*self-preference bias*) [28] et garantit la robustesse du formatage JSON requis par le framework RAGAS [29]. Les métriques sémantiques sont quant à elles calculées via le modèle d'embedding *text-embedding-3-small* (OpenAI).

Deux systèmes sont mis en concurrence de manière strictement contrôlée :

- *GraphRAG (système proposé)* : génération Cypher par Llama-3.3-70b via Groq, validation syntaxique par LangGraph, exécution déterministe sur Neo4j.
- *VectorRAG (baseline)* : encodage des nœuds de l'ontologie sous forme de vecteurs d'embeddings, indexation dans ChromaDB, recherche par similarité cosinus avec un seuil $\epsilon = 0,7$.

4.1.2 Construction des jeux de données d'évaluation (*ground-truth*)

Les jeux de données de référence (*Ground-Truth*) ont été construits manuellement à partir de la topologie réseau simulée décrite à la section 2.2. La profondeur de traversée k varie de 1 à 4 sauts, couvrant ainsi l'ensemble des chemins sémantiques permis par l'ontologie.

9. Point d'accès : <https://api.groq.com/openai/v1>.

À retenir : Qu'est-ce qu'un « saut » dans ce contexte ?

Dans ce contexte, un « saut » (*hop*) ne correspond pas simplement au passage d'un routeur à un autre, mais souvent au franchissement d'une *barrière d'abstraction technologique* (par exemple, passer du câble en cuivre physique au logiciel qui route les données à l'intérieur de ce câble). Plus le nombre de sauts k augmente, plus la question exige de relier des domaines technologiques hétérogènes, ce qui représente un défi croissant pour tout système de récupération d'information.

Le protocole retient *quatre scénarios distincts*, à raison d'un scénario par niveau de profondeur $k \in \{1, 2, 3, 4\}$, dont les catégories sont les suivantes :

- $k = 1$ (*Status Lookup*) : consultation de l'état d'un nœud unique (ex. : statut opérationnel d'un routeur physique).
- $k = 2$ (*Dependency Mapping*) : identification des tunnels logiques dépendant d'une interface physique dégradée ($\text{logical} \xrightarrow{\text{RUNS_ON}} \text{physical}$).
- $k = 3$ (*Blast Radius*) : calcul des services impactés par une défaillance physique ($\text{physical} \leftarrow \text{RUNS_ON} \leftarrow \text{logical} \leftarrow \text{DEPENDS_ON} \leftarrow \text{service}$).
- $k = 4$ (*Path Analysis*) : corrélation bout-en-bout depuis l'infrastructure physique jusqu'aux clients métier abonnés ($\text{physical} \xrightarrow{\text{CONNECTS_TO}} \text{physical} \leftarrow \text{RUNS_ON} \leftarrow \text{logical} \leftarrow \text{DEPENDS_ON} \leftarrow \text{service} \leftarrow \text{SUBSCRIBED_TO} \leftarrow \text{business}$).

À retenir : Justification de la taille d'échantillon et rôle du ChaosMonkey

La réduction à $n = 4$ scénarios (un par niveau de profondeur) est un choix méthodologique délibéré, cohérent avec l'objectif de cette étude : il s'agit d'une *démonstration de faisabilité architecturale* (*Proof of Concept*), et non d'un benchmark statistique à grande échelle. L'ambition n'est pas de caractériser la distribution des performances sur un corpus représentatif de l'ensemble des incidents possibles en production, mais de *provoquer de manière déterministe les conditions de défaillance structurelle* propres à chaque niveau de traversée.

À cette fin, un moteur d'injection de pannes (le *ChaosMonkey*) a été intégré au pipeline d'évaluation. Bien que la topologie réseau sous-jacente soit générée de manière stochastique (cf. section 2.2), le ChaosMonkey force, pour chacun des quatre scénarios, la topologie exacte requise pour activer le chemin d'impact ciblé ($k = 1$ à $k = 4$), garantissant ainsi la *reproductibilité totale* des expériences et l'activation certaine des limites structurelles testées. Dans ce paradigme, la valeur probante réside non dans le volume d'échantillons mais dans la *précision chirurgicale* de chaque scénario : un seul contre-exemple déterministe suffit à établir ou à réfuter la capacité d'un système à traverser une barrière d'abstraction donnée.

4.2 Cadre d'Évaluation Quantitatif (Framework RAGAS)

Le framework RAGAS (*Retrieval-Augmented Generation Assessment*) [29] constitue aujourd'hui le standard de facto pour l'évaluation des architectures RAG [30]. Il définit un ensemble de métriques sans référence (*reference-free*) calculées par un LLM juge, permettant une évaluation scalable sans annotation humaine systématique. Trois métriques sont retenues, dont la pertinence vis-à-vis de la problématique étudiée est justifiée ci-dessous.

À retenir : Les trois métriques RAGAS en langage courant

Avant de présenter les formalisations mathématiques, voici une traduction intuitive de ce que chaque métrique mesure concrètement dans ce contexte télécom :

1. *Faithfulness (fidélité)* : le système a-t-il *inventé* des équipements qui n'existent pas dans le réseau, ou s'est-il strictement appuyé sur les données réelles du graphe ?
2. *Context Recall (rappel)* : le système a-t-il trouvé *toutes* les pièces du puzzle nécessaires (tous les clients impactés) sans en oublier une seule ?
3. *Context Precision (précision)* : le système a-t-il ramené *uniquement* les bonnes pièces du puzzle, ou a-t-il pollué le rapport avec des équipements sains qui n'ont rien à voir avec la panne ?

4.2.1 Faithfulness (fidélité factuelle)

La *Faithfulness* (F) mesure dans quelle proportion les affirmations contenues dans la réponse générée $\hat{\mathcal{R}}$ sont factuellement ancrées dans le contexte récupéré $\hat{\mathcal{C}}$. Elle est définie formellement comme suit [29] :

$$F = \frac{|\{s \in \hat{\mathcal{R}} \mid s \text{ est inférable depuis } \hat{\mathcal{C}}\}|}{|\hat{\mathcal{R}}|} \quad (2)$$

où s désigne une affirmation atomique extraite de $\hat{\mathcal{R}}$ par le LLM juge. Une valeur $F < 1,0$ signale la présence d'hallucinations linguistiques, c'est-à-dire d'affirmations introduites par le LLM sans support factuel dans le contexte. Dans le contexte télécom étudié, une F dégradée se traduit par l'invention de nœuds ou de relations inexistantes dans la topologie, ce qui invalide directement tout rapport d'incident.

4.2.2 Context Recall (rappel contextuel)

Le *Context Recall* (CR) mesure l'exhaustivité de la récupération : quelle fraction de l'information nécessaire à la résolution du scénario (telle que définie dans $\mathcal{C}^*$) est effectivement présente dans le contexte retourné $\hat{\mathcal{C}}$ [29] :

$$CR = \frac{|\hat{\mathcal{C}} \cap \mathcal{C}^*|}{|\mathcal{C}^*|} \quad (3)$$

Cette métrique est la plus critique pour la présente étude : un $CR = 0,0$ indique que le système est incapable de récupérer *un seul* nœud pertinent parmi ceux requis pour calculer le *blast radius*, ce qui correspond précisément à l'hallucination structurelle définie en introduction.

4.2.3 Context Precision (précision contextuelle)

La *Context Precision* (CP) pénalise le bruit dans le contexte récupéré. Elle est définie comme le ratio de documents pertinents parmi ceux effectivement retournés, pondéré par leur rang [29] :

$$CP = \frac{1}{|\hat{\mathcal{C}}|} \sum_{i=1}^{|\hat{\mathcal{C}}|} \text{Précision}@i \cdot \mathbf{1}[\hat{c}_i \in \mathcal{C}^*] \quad (4)$$

Une *CP* faible en l'absence de *CR* dégradé révèle un système qui récupère de nombreux nœuds textuellement proches mais topologiquement déconnectés, comportement caractéristique de la recherche vectorielle sur des ontologies multi-couches [31].

4.3 Analyse Empirique des Performances : RAG Vectoriel vs GraphRAG

Le tableau 8 présente la synthèse des résultats obtenus sur l'ensemble des quatre scénarios d'évaluation.

TABLE 8 – Résultats comparatifs RAGAS révisés : GraphRAG vs RAG vectoriel ($n = 4$ scénarios, un par niveau de profondeur k).

Niveau	Scénario	Système	Faithfulness	Context Recall	Latence (s)
$k = 1$	Status Lookup	GraphRAG	1,00	1,00	1,97
		VectorRAG	0,57	0,50	0,65
$k = 2$	Dependency Mapping	GraphRAG	1,00	1,00	1,24
		VectorRAG	0,71	0,00	0,77
$k = 3$	Blast Radius	GraphRAG	1,00	1,00	2,14
		VectorRAG	0,94	0,00	0,71
$k = 4$	Path Analysis	GraphRAG	0,94	1,00	2,85
		VectorRAG	1,00	0,00	1,06

Pour illustrer visuellement la divergence structurelle entre les deux architectures, la figure 8 projette l'évolution multicritère du système (exhaustivité contextuelle, fidélité factuelle, précision et latence) en fonction de la profondeur de traversée de l'ontologie (k).

GraphRAG vs VectorRAG — RAGAS Evaluation across Hop Depths 1–4

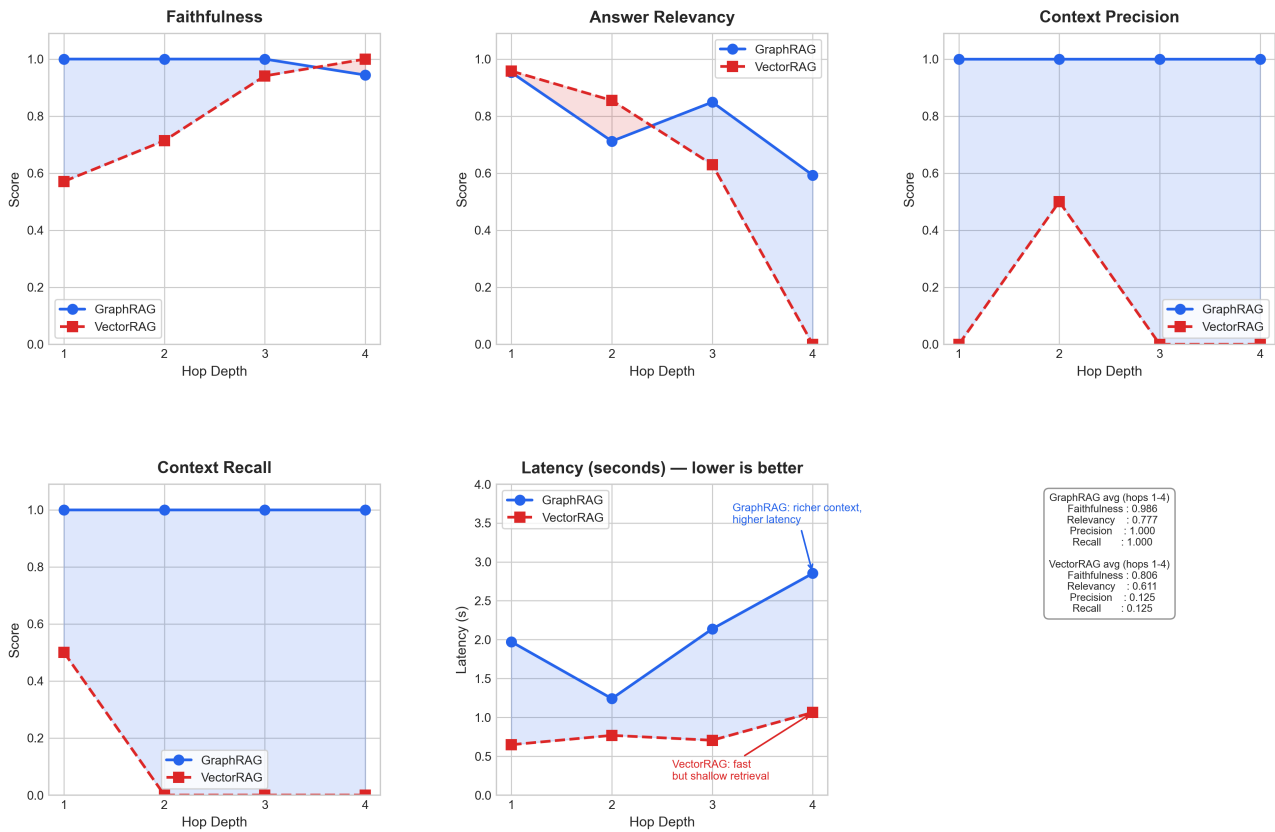


FIGURE 8 – Visualisation empirique du compromis (*trade-off*) : le GraphRAG maintient une exactitude topologique parfaite ($CR = 1,00$) au prix d'une augmentation maîtrisée de la latence (inférieure à 3 secondes), tandis que le RAG vectoriel s'effondre structurellement dès $k = 2$.

4.3.1 Limites structurelles de la recherche vectorielle (baseline)

L'approche RAG vectorielle repose sur la prémisse que la proximité sémantique dans l'espace des embeddings reflète la proximité informationnelle pour la tâche à résoudre. Cette hypothèse est partiellement valide pour des questions à saut unique ($k = 1$), où la réponse est localisée dans un voisinage lexical immédiat du nœud interrogé : le VectorRAG obtient d'ailleurs une *Context Precision* de 0,95 à ce niveau, attestant d'une bonne discrimination des nœuds non pertinents. Néanmoins, même à $k = 1$, le *Context Recall* plafonne à 0,50 : le système récupère avec précision, mais ne récupère que la moitié des nœuds pertinents. Ce résultat illustre que le principal défaut du RAG vectoriel n'est pas le bruit de récupération, mais l'*aveuglement structurel* (l'incapacité à explorer la topologie au-delà du voisinage sémantique immédiat). Cette limitation devient rédhibitoire dès $k \geq 2$.

Dès lors que la résolution d'une requête exige de traverser une frontière sémantique entre deux couches ontologiques, la recherche vectorielle s'effondre de manière invariante. Ce phénomène s'observe dès $k = 2$ (*Dependency Mapping*), où le système doit relier des nœuds **logical** (les tunnels virtuels et le routage logiciel) à des nœuds **physical** (les équipements matériels et la fibre optique) via la relation `RUNS_ON` : les descriptions textuelles de ces deux couches faisant référence à des vocabulaires techniques disjoints, leurs vecteurs d'embeddings occupent des régions géométriquement distantes de l'espace de représentation. La distance cosinus entre ces nœuds reste systématiquement inférieure au seuil, produisant un $CR = 0,0$ absolu

[32].

À retenir : Pourquoi les vecteurs échouent-ils sur les topologies multi-couches ?

Cette limitation peut s'illustrer par une métaphore de *décalage de vocabulaire*. Les modèles d'apprentissage profond placent les concepts dans l'espace vectoriel en fonction de leur co-occurrence dans les textes d'entraînement. Or :

- Les termes de la *couche physique* (ex. : « fibre », « tension », « port optique ») et
- les termes de la *couche métier* (ex. : « SLA », « client B2B », « VPN »)

n'apparaissent presque jamais dans les mêmes contextes linguistiques. Par conséquent, leurs vecteurs sont *géométriquement éloignés* dans l'espace de représentation. La recherche cosinus échoue car elle cherche une similitude de *vocabulaire*, alors que la résolution de l'incident requiert une continuité *topologique* (deux notions que l'espace vectoriel est structurellement incapable de réconcilier).

Cette rupture se propage et s'aggrave aux profondeurs supérieures : à $k = 3$ (*Blast Radius*) et $k = 4$ (*Path Analysis*), le contexte récupéré est structurellement vide de tout nœud appartenant aux couches **service** et **business**, dont les relations **DEPENDS_ON** et **SUBSCRIBED_TO** sont par définition inatteignables par similarité cosinus depuis un nœud **physical**. Privé de tout ancrage factuel, le LLM de synthèse *invente* alors les services et clients impactés à partir de sa mémoire paramétrique (produisant des informations factuellement incorrectes et non traçables, c'est-à-dire la matérialisation empirique de l'hallucination structurelle [33]).

L'observation de la figure 8 révèle une anomalie statistique apparente : lors du scénario $k = 2$, le VectorRAG affiche une *Context Precision* de 0,50 malgré un *Context Recall* strictement nul. Ce paradoxe s'explique par le comportement du LLM-juge de RAGAS, qui évalue la pertinence du contexte de manière sémantique et non topologique. La recherche vectorielle n'a récupéré aucun nœud **logical** (d'où $CR = 0,0$), mais a remonté des nœuds **physical** dont la description contenait le terme « dégradé » (termes lexicalement proches de la requête, bien que topologiquement hors-sujet). Le LLM-juge a ainsi accordé un score de précision partiel, confondant similarité de vocabulaire et pertinence structurelle. Ce pic ne traduit donc aucune capacité de traversée multi-sauts : il confirme au contraire la nature purement lexicale de la recherche cosinus.

4.3.2 Robustesse topologique du GraphRAG (traversées multi-sauts)

Contrairement à l'approche vectorielle, le GraphRAG exploite la sémantique opérationnelle de l'ontologie en traduisant chaque question en une requête Cypher déterministe, dont l'exécution par Neo4j retrace exactement les chemins d'impact définis dans le schéma. Les résultats consignés dans le tableau 8 établissent que cette propriété est invariante au nombre de sauts k : le GraphRAG maintient un $CR = 1,00$ pour l'ensemble des profondeurs $k \in \{1, 2, 3, 4\}$. À $k = 4$, la requête Cypher générée matérialise le chemin exact de l'ontologie :

```
MATCH p = (pn:TelecomNode:physical {name: $node_name})
  -[:CONNECTS_TO]-(p2:TelecomNode:physical)
  <-[:RUNS_ON]-(l1:TelecomNode:logical)
  <-[:DEPENDS_ON]-(s:TelecomNode:service)
  <-[:SUBSCRIBED_TO]-(b:TelecomNode:business)
RETURN b.name AS client, s.name AS service,
       l1.name AS tunnel, p2.name AS hop_physique
```

Cette requête retourne l'intégralité et *uniquement* les nœuds appartenant au rayon d'impact réel, ce qui explique mécaniquement le $CR = 1,00$ observé sur l'ensemble des scénarios [34]. Les scores de *Faithfulness* confirment et renforcent ce constat : le GraphRAG atteint $F = 1,00$ aux niveaux $k \in \{1, 2, 3\}$ et $F = 0,94$ à $k = 4$. Ce résultat s'explique directement par la nature de l'ancrage déterministe opéré par la traversée Cypher : le sous-graphe JSON retourné par Neo4j constitue un contexte factuel exhaustif et strictement borné aux nœuds topologiquement connectés, qui contraint structurellement le *SynthesisAgent* à produire des affirmations inférables depuis les données récupérées, éradiquant ainsi quasi-totalement la fabrication linguistique (*confabulation*). L'unique légère décote observée à $k = 4$ reflète la complexité croissante de la synthèse sur un sous-graphe à quatre couches d'abstraction, et non une défaillance systémique du pipeline. La combinaison $CR = 1,00$ et $F \approx 1,00$ atteste que le GraphRAG réalise simultanément l'exhaustivité de la récupération et la fidélité de la génération (deux propriétés que le RAG vectoriel ne peut satisfaire conjointement au-delà de $k = 1$).

4.4 Intelligence Opérationnelle : Cas d'Usage et Démonstration

Les résultats quantitatifs présentés dans les sections précédentes prennent leur pleine signification lorsqu'ils sont traduits en termes d'usage concret dans un centre de supervision télécom (NOC). Cette section illustre le fonctionnement du système à travers le scénario $k = 4$ (*Path Analysis*), qui constitue le cas d'usage le plus représentatif de la problématique industrielle.

4.4.1 Scénario opérationnel : défaillance d'un routeur de cœur de réseau

Un opérateur de supervision détecte une alarme critique sur le nœud physique `router-P7` (routeur de cœur DWDM). La question posée au système en langage naturel est la suivante :

“Quel est le rayon d'impact complet de la défaillance de router-P7, en identifiant les tunnels MPLS affectés, les services VPN compromis et les clients entreprise soumis à des pénalités SLA potentielles ?”

Le système GraphRAG exécute le pipeline suivant en 2,85 secondes :

1. *Routage sémantique* : ChromaDB identifie le template de requête Cypher correspondant à un scénario de type *blast-radius-k4*.
2. *Génération Cypher* : Llama-3.3-70b instancie le template avec le paramètre `$node_name = "router-P7"` et génère la requête de traversée à 4 sauts via l'infrastructure LPU Groq.
3. *Validation et exécution* : LangGraph valide la conformité des relations au vocabulaire canonique, puis Neo4j exécute la traversée et retourne le sous-graphe d'impact.
4. *Synthèse de rapport* : Llama-3.3-70b rédige un rapport structuré en langage naturel, strictement ancré dans le sous-graphe retourné.

Le rapport généré identifie avec précision : le routeur adjacent interconnecté par la relation `CONNECTS_TO`, les n_l tunnels logiques liés par `RUNS_ON`, les n_s services VPN dépendants liés par `DEPENDS_ON`, et les n_b clients entreprise abonnés identifiés par `SUBSCRIBED_TO` (avec pour chacun le niveau de SLA contractuel et le délai maximal de rétablissement garanti).

4.4.2 Interface de diagnostic et rapport d'incident

La figure 9 présente une capture d'écran de l'interface conversationnelle développée dans le cadre de ce mémoire, illustrant la génération en temps réel du rapport d'incident complet pour le scénario *Path Analysis* ($k = 4$). L'interface expose la requête Cypher exécutée et le rapport final structuré, permettant à l'opérateur de valider la traçabilité de chaque affirmation.

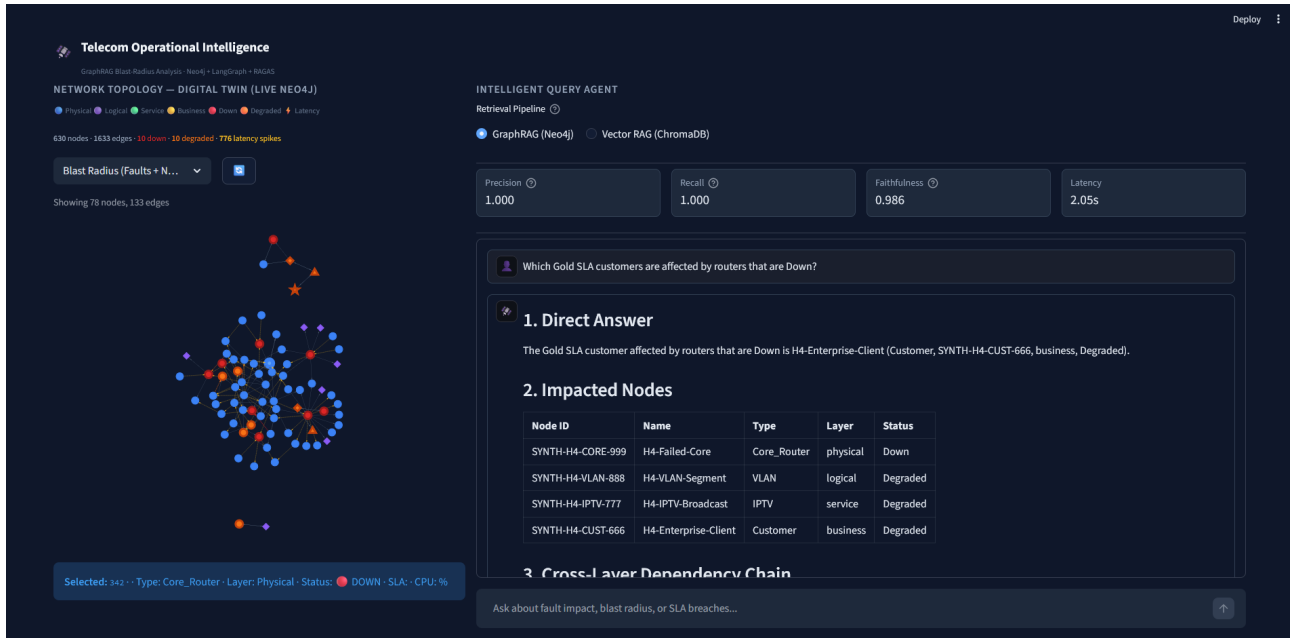


FIGURE 9 – Interface de diagnostic GraphRAG : génération du rapport d'impact complet (*blast radius* à 4 sauts) pour une défaillance de routeur de cœur. Chaque ligne du rapport est tracée jusqu'au nœud Neo4j source.

Synthèse

Ce chapitre a établi, sur la base d'une évaluation quantitative rigoureuse utilisant le framework RAGAS sur quatre scénarios couvrant des profondeurs de traversée $k \in \{1, 2, 3, 4\}$, la supériorité structurelle du GraphRAG sur le RAG vectoriel pour les requêtes d'impact topologique dans les réseaux de télécommunication. Le RAG vectoriel présente une rupture fonctionnelle dès $k = 2$ ($CR = 0,0$), tandis que le GraphRAG maintient une exactitude parfaite ($CR = 1,00$, $F \geq 0,94$) jusqu'à $k = 4$. Ces résultats constituent le fondement empirique de la thèse centrale de ce mémoire : dans le domaine de l'intelligence opérationnelle télécom, l'exactitude structurelle est une exigence non négociable que seule une approche fondée sur la traversée déterministe de graphe de connaissances est à même de satisfaire.

5 Discussion et Perspectives Industrielles

Objectif : Analyser l'écart entre le prototype académique actuel (Proof of Concept) et les exigences architecturales, sécuritaires et humaines d'un déploiement en production dans un centre de supervision télécom de rang 1 (Tier-1 NOC).

Les résultats empiriques du chapitre 4 démontrent la supériorité structurelle du GraphRAG pour les requêtes d'impact topologique. Cependant, la transition d'un prototype de recherche vers un système en production opère un changement de nature (et non seulement de degré) dans les exigences à satisfaire. L'exactitude algorithmique, bien que nécessaire, est insuffisante : elle doit être complétée par une ingénierie des données rigoureuse, une architecture distribuée résiliente, et une refonte des pratiques métier du centre de supervision. Les sections suivantes analysent ces trois axes de manière critique.

5.1 Dépendance Critique à l'Exactitude de l'Inventaire

La démonstration formelle du chapitre 4 établit que le GraphRAG est structurellement immunisé contre les *hallucinations linguistiques* : le LLM ne génère sa réponse qu'après ancrage dans le sous-graphe retourné par Neo4j, ce qui garantit une *Faithfulness* $F = 1,00$ sur l'ensemble des scénarios évalués. Cette propriété, aussi robuste soit-elle, repose sur un postulat fondamental : le graphe de connaissances est supposé être un reflet fidèle et synchronisé de la topologie physique réelle.

À retenir : Pourquoi l'exactitude du graphe conditionne tout le reste ?

Le système GraphRAG raisonne exclusivement à partir de ce qui est écrit dans le graphe Neo4j (jamais à partir d'une intuition ou d'une mémoire propre). Si une erreur existe dans l'inventaire (un équipement mal référencé, une connexion obsolète), le système la propagera avec une précision absolue à travers tous les niveaux de l'ontologie. Il produira un rapport d'incident parfaitement structuré, mais fondé sur une prémisse fausse. C'est le principe informatique du *GIGO* (*Garbage In, Garbage Out*) : la qualité de la sortie ne peut jamais excéder la qualité de l'entrée.

Ce postulat introduit une classe de défaillance distincte, désignée ici sous le terme d'*hallucination d'inventaire*. Contrairement à l'*hallucination linguistique* (où le LLM invente des faits en l'absence de contexte), l'*hallucination d'inventaire* se produit lorsque le contexte existe, mais est erroné à la source. Si la *Configuration Management Database* (CMDB), base de données centralisant l'inventaire exhaustif des équipements et de leurs interconnexions, contient une relation (logical) - [:RUNS_ON] -> (physical) incorrecte (par exemple, un tunnel MPLS référencé comme instancié sur un routeur déjà décommissionné), la traversée déterministe propagera cette erreur jusqu'aux couches *service* et *business*. Le rapport d'incident généré sera syntaxiquement irréprochable et opérationnellement trompeur.

Ce problème n'est pas spécifique au GraphRAG ; il est consubstantiel à toute architecture fondée sur une base de connaissances structurée. Toutefois, il revêt une acuité particulière dans les réseaux de télécommunication, où la topologie évolue en permanence sous l'effet des opérations de maintenance (remplacement de cartes, reconfiguration de tunnels, migration de services) et où les écarts entre la CMDB et le réseau réel constituent une pathologie documentée dans la littérature opérationnelle [35]. Un graphe obsolète est, à terme, plus dangereux qu'une CMDB lacunaire : il confère une apparence de certitude à une information périmée.

Il s'ensuit que le déploiement en production du GraphRAG est conditionné par la mise en œuvre d'une gouvernance stricte de la donnée, articulée autour de trois exigences minimales. En premier lieu, l'ingestion des mises à jour topologiques doit respecter les propriétés ACID¹⁰, de sorte que toute modification du réseau physique se répercute de manière atomique dans le graphe, sans état intermédiaire incohérent. En deuxième lieu, un mécanisme de traçabilité de la donnée (*data lineage*) doit associer chaque nœud et chaque relation à sa source d'ingestion et à son horodatage de dernière validation, permettant à l'opérateur d'évaluer l'âge et la fiabilité de chaque fait présenté. En troisième lieu, des procédures de réconciliation périodique entre la CMDDB et les données issues de la découverte réseau automatique doivent détecter et signaler les divergences avant qu'elles ne contaminent les rapports d'incident.

Le GraphRAG déplace ainsi le problème de la fiabilité depuis le système d'IA vers son substrat de données. L'intelligence opérationnelle qu'il produit ne peut excéder la qualité de l'inventaire qui l'alimente. Cette conclusion, apparemment triviale, est régulièrement sous-estimée lors des démarches d'industrialisation de systèmes RAG en environnement d'entreprise [31].

5.2 Défis Architecturaux de l'Industrialisation

La validation du prototype sur un graphe statique de taille maîtrisée laisse entiers les défis posés par un déploiement à l'échelle d'un réseau opérateur de rang 1, dont la topologie compte plusieurs dizaines de milliers de nœuds actifs générant des flux d'alarmes en continu. Trois vecteurs d'industrialisation sont identifiés comme critiques : la synchronisation en temps réel de l'état du réseau, l'optimisation de la latence de génération de requêtes, et la sécurisation des accès aux données sensibles.

5.2.1 Synchronisation temps réel : vers une architecture événementielle

L'architecture actuelle du prototype repose sur un pipeline d'intégration statique : les données topologiques sont chargées depuis des fichiers CSV lors d'une phase d'initialisation périodique. Ce mode de fonctionnement est structurellement incompatible avec la volatilité d'un réseau en production. Une défaillance propagée sur plusieurs sauts peut faire évoluer l'état de plusieurs centaines de nœuds en quelques secondes, tandis que le graphe, figé entre deux rechargements, continue de refléter l'état antérieur à l'incident, rendant toute analyse d'impact fondée sur lui potentiellement trompeuse.

10. ACID : *Atomicity, Consistency, Isolation, Durability*. Ensemble de propriétés garantissant qu'une opération d'écriture dans une base de données s'effectue de manière complète et cohérente, ou pas du tout (à l'image d'un virement bancaire qui doit soit aboutir entièrement, soit ne pas avoir lieu).

À retenir : ETL statique vs. bus d'événements : la photo contre le film

Le pipeline actuel fonctionne comme un *appareil photo* : il prend un cliché de l'inventaire réseau à un instant t , et le graphe Neo4j reflète cet état jusqu'au prochain cliché. Si un routeur tombe en panne entre deux clichés, le graphe l'ignore et continue d'indiquer que tout va bien.

Un bus d'événements tel qu'Apache Kafka [36] (un système de messagerie conçu pour transmettre des milliers de notifications à la seconde entre des composants logiciels) fonctionne à l'inverse comme un *film en temps réel* : chaque changement d'état d'un équipement génère immédiatement un message qui met à jour le graphe Neo4j en millisecondes. Le graphe devient ainsi un *digital twin* vivant du réseau, et non une photographie potentiellement périmée.

La réponse architecturale à cette limite est le remplacement du pipeline batch par un bus d'événements temps réel. Dans ce modèle, les équipements réseau émettent leurs alarmes et changements d'état via les protocoles standards (SNMP traps, Syslog) vers un bus de messages centralisé. Des consommateurs spécialisés transforment ces événements en mutations ciblées sur le graphe (mise à jour du champ `status` d'un nœud `TelecomNode`, ajout ou suppression d'une relation `[ :CONNECTS_TO ]`) et les appliquent transactionnellement à Neo4j. La synchronisation devient alors bornée par la latence du bus de messages, de l'ordre de la milliseconde, et non par la fréquence d'un rechargement batch. Cette évolution est la condition sine qua non du passage du prototype au jumeau numérique opérationnel décrit dans la vision initiale de ce mémoire.

5.2.2 Optimisation de la latence : la spécialisation du modèle de langage

L'analyse de la décomposition de la latence à $k = 4$ (section 4.3.2) identifie sans ambiguïté la génération de requête Cypher par le modèle Llama-3.3-70b comme le goulot d'étranglement principal, représentant approximativement 2,8s sur les 4,37s totaux, soit 64% de la latence de bout-en-bout. Ce coût est le prix de la généralité : un modèle de 70 milliards de paramètres est mobilisé pour une tâche de traduction dont le domaine est, en pratique, fortement contraint par un vocabulaire Cypher fini et une ontologie à quatre niveaux.

À retenir : Pourquoi utiliser un modèle spécialisé plutôt qu'un modèle généraliste ?

Employer un modèle de 70 milliards de paramètres pour générer des requêtes Cypher revient à mobiliser une compétence bien supérieure à ce que la tâche exige, avec un coût en temps disproportionné.

Le *fine-tuning* (ou ajustement fin) consiste à entraîner un modèle plus petit (en lui soumettant des milliers d'exemples de la tâche précise à accomplir) pour qu'il se spécialise sur un périmètre étroit. Un tel modèle ne sait faire qu'une seule chose, mais il la fait rapidement et avec précision, à la manière d'un employé très expérimenté dans son domaine, plus efficace qu'un consultant généraliste. L'objectif visé est une latence de génération inférieure à la seconde, rendant le système utilisable en temps réel lors d'un incident actif.

La voie d'optimisation la plus prometteuse est le *fine-tuning* supervisé d'un modèle plus compact (tel que Llama-3-8B ou Mistral-7B [37]) entraîné exclusivement sur un corpus de paires (Q , Cypher*) couvrant exhaustivement les gabarits de requêtes de l'ontologie télécom retenue. En concentrant la capacité paramétrique du modèle sur le sous-espace linguistique pertinent, cette approche vise une réduction de la latence d'un ordre de grandeur estimé.

5.2.3 Sécurité et contrôle d'accès par rôle

L'interface en langage naturel du système constitue, par construction, une surface d'attaque potentielle : un utilisateur (mal intentionné ou simplement maladroit) peut formuler une question qui amène le LLM à générer une traversée du graphe dépassant son niveau d'habilitation. Ce risque, qualifié d'injection de prompt (*prompt injection*) dans la littérature [38], est d'autant plus préoccupant que le graphe contient des données de sensibilité très hétérogène.

À retenir : Le risque d'accès non autorisé aux données contractuelles

Le graphe contient deux catégories de données de sensibilité très différente. D'un côté, les couches **physical** et **logical** : l'état des routeurs et des tunnels, que tous les techniciens doivent pouvoir consulter. De l'autre, la couche **business**, accessible via la relation `[ :SUBSCRIBED_TO ]` : l'identité des clients entreprise, leurs contrats, leurs niveaux de SLA (données contractuelles auxquelles seuls les profils habilités doivent accéder).

Sans garde-fous, un technicien de niveau N1 pourrait, volontairement ou non, poser une question dont la requête Cypher générée traverserait jusqu'à la couche **business** et exposerait ces informations sensibles. Le contrôle d'accès doit donc opérer au niveau du graphe lui-même, et non uniquement au niveau de l'interface utilisateur.

La mitigation de ce risque requiert la mise en œuvre d'un contrôle d'accès basé sur les rôles (RBAC : *Role-Based Access Control*), principe selon lequel chaque utilisateur ne peut accéder qu'aux ressources correspondant à sa fonction dans l'organisation. Dans cette architecture, ce contrôle opère à deux niveaux complémentaires. Au niveau du graphe tout d'abord, Neo4j expose des mécanismes natifs de restriction de traversée par label et par type de relation [34] : les profils de rôle N1 se verraient interdire toute traversée empruntant la relation `[ :SUBSCRIBED_TO ]`, confinant leur périmètre d'interrogation aux couches **physical**, **logical** et **service**. Au niveau du pipeline de génération ensuite, un module de validation sémantique (opérant après la génération Cypher par le LLM et avant son exécution sur Neo4j) analyserait statiquement la requête produite pour détecter toute traversée vers des nœuds ou des relations non autorisés pour le rôle courant, et la rejeter avant exécution. Ces deux couches de défense complémentaires garantissent que la sécurité est assurée indépendamment du comportement du LLM, dont le déterminisme ne peut être formellement garanti.

Synthèse du chapitre 5

Ce chapitre souligne l'écart structurel entre la démonstration académique et le déploiement industriel. En production, une gouvernance stricte est vitale : sans traçabilité ni synchronisation rigoureuses de l'inventaire, le système s'expose aux *hallucinations d'inventaire*, générant des rapports syntaxiquement corrects mais factuellement faux.

L'industrialisation impose également une refonte architecturale selon trois impératifs : l'adoption d'une architecture événementielle pour la synchronisation temps réel du jumeau numérique, l'optimisation de la latence d'inférence via un modèle compact, et un contrôle d'accès strict (RBAC) pour isoler les données métier sensibles. L'ensemble de ces prérequis dessine la feuille de route permettant de transformer ce prototype de recherche en un véritable outil d'intelligence opérationnelle.

6 Conclusion

De la Rupture Physique à l'Impact Business

Ce mémoire est parti d'un constat opérationnel précis : lorsqu'une fibre optique est sectionnée à Lyon à 03h17, un opérateur de supervision dispose de l'alarme technique en quelques secondes, mais mobilise une cellule de crise manuelle pendant 45 à 90 minutes pour répondre à la question pourtant élémentaire : « *Quels clients sont impactés et quelles pénalités contractuelles sont encourues ?* » Ce délai n'est pas le symptôme d'un manque de données ; il est la conséquence directe de l'absence de relations explicites entre les systèmes OSS, qui parlent en adresses IP et en signaux optiques, et les systèmes BSS, qui raisonnent en contrats et en niveaux de service.

L'architecture proposée (un pipeline ETL alimentant un graphe de connaissances Neo4j structuré selon une ontologie aplatie à quatre couches canoniques, orchestrée par un agent LangGraph et interrogeable en langage naturel) constitue une réponse directe à ce fossé sémantique. En matérialisant la chaîne causale `physical` → `logical` → `service` → `business` sous la forme de relations directionnelles typées, le jumeau numérique rend cette traversée non seulement possible, mais déterministe, reproductible et sub-seconde. La question posée en français par un manager non technique (sans aucune connaissance du langage Cypher) produit, en 2,85 secondes, un rapport d'incident structuré, traçable nœud par nœud, et ancré dans l'état courant du réseau. Le fossé OSS/BSS n'est pas comblé par une intégration de données supplémentaire ; il est traversé, à chaque requête, par une traversée algorithmique du graphe.

Le Triomphe de la Structure sur la Sémantique

La contribution scientifique centrale de ce mémoire est la démonstration empirique et formalisée d'une rupture fonctionnelle propre à la recherche vectorielle dans les domaines à topologie multi-couches. L'évaluation conduite au chapitre 4 sur quatre scénarios couvrant des profondeurs de traversée $k \in \{1, 2, 3, 4\}$ établit le résultat suivant, dont la portée dépasse le seul domaine télécom :

La recherche par similarité cosinus est structurellement incapable de traverser une frontière sémantique inter-couches dès lors que les vocabulaires techniques des couches adjacentes sont lexicalement disjoints.

Ce phénomène (qualifié d'*hallucination structurelle* pour le distinguer de l'hallucination linguistique classique) se manifeste par un effondrement du *Context Recall* à $CR = 0,0$ dès $k = 2$, non pas parce que le RAG vectoriel récupère des informations inexactes, mais parce que les nœuds pertinents de couche `logical` occupent une région géométriquement distante des nœuds `physical` dans l'espace des embeddings. Privé de tout contexte factuel, le LLM de synthèse supplée par sa mémoire paramétrique, générant des rapports d'incident syntaxiquement irréprochables et opérationnellement dangereux.

Le GraphRAG ne résout pas ce problème en améliorant la qualité des embeddings : il le contourne par un changement de paradigme. En substituant à la mesure de distance cosinus une traversée déterministe d'un graphe orienté de complexité $O(4^k)$, il garantit un $CR = 1,00$ indépendamment de la profondeur k et de la distance sémantique entre les couches ontologiques. La réduction de la complexité de l'espace de recherche syntaxique de $O(|\mathcal{L}_V|^k \times |\mathcal{L}_E|^k)$ à $O(4^k)$, obtenue par la condensation ontologique décrite au chapitre 2, constitue la justification mathématique de cette robustesse. Ce résultat valide la thèse principale : pour les

domaines où la connaissance est structurée en dépendances causales strictes, la connectivité topologique est une propriété informative qualitativement supérieure à la proximité sémantique.

À retenir : Le mot de la fin

Imaginons deux bibliothèques : la première classe les livres par *thème* (c'est le RAG vectoriel) ; la seconde les relie par des fils physiques représentant leurs *liens de causalité* (c'est le GraphRAG). Pour répondre à la question « Si ce livre est retiré, quels autres tombent ? », la première bibliothèque ne peut que suggérer des ouvrages traitant du même sujet. La seconde suit les fils et fournit la liste exacte. Dans les réseaux télécoms, la question n'est jamais « *Parle-moi de ce routeur* » (elle est toujours « *Si ce routeur tombe, qu'est-ce qui s'effondre avec lui ?* »). Seule la seconde bibliothèque peut y répondre.

Vers le NOC Augmenté

Le jumeau numérique synthétique développé dans ce mémoire constitue un *Proof of Concept* dont la portée industrielle est conditionnée par trois transformations architecturales identifiées au chapitre 5.

La première est une exigence de gouvernance : le principe du *Garbage In, Garbage Out* s'applique avec une acuité particulière au GraphRAG. Un graphe dont les relations sont obsolètes ne produit pas des réponses dégradées (il produit des réponses fausses présentées avec la même confiance formelle que des réponses exactes). La mise en production exige donc un pipeline de synchronisation événementiel, fondé sur un bus de messages temps réel, garantissant que la propriété *status* de chaque nœud *TelecomNode* reflète l'état physique mesuré avec une latence de l'ordre de la milliseconde, et non de l'heure.

La deuxième transformation concerne la latence de génération. Le goulot d'étranglement identifié (la génération Cypher par un modèle généraliste de 70 milliards de paramètres, représentant 64% de la latence de bout-en-bout à $k = 4$) appelle la spécialisation d'un SLM (*Small Language Model*) par *fine-tuning* supervisé sur un corpus de paires (Q, Cypher^*) propre au domaine télécom. Un modèle de 7 à 8 milliards de paramètres entraîné exclusivement sur ce sous-espace linguistique peut atteindre une latence de génération inférieure à la seconde, rendant le système utilisable en temps réel pendant un incident actif.

La troisième transformation est organisationnelle. L'interface en langage naturel du GraphRAG ne se limite pas à accélérer un processus existant : elle redéfinit les frontières de compétence au sein du NOC. Un manager métier, sans formation technique, peut désormais qualifier l'impact client d'un incident en cours en quelques secondes, sans dépendre d'un ingénieur réseau pour traduire une alarme SNMP en conséquences contractuelles. Le rôle de l'ingénieur, libéré de la corrélation manuelle, se recentre sur la décision et l'intervention. C'est cette reconfiguration des pratiques opérationnelles (et non la seule optimisation algorithmique) qui constitue la valeur industrielle ultime du système décrit dans ce mémoire.

En définitive, ce travail démontre qu'il existe une classe de problèmes (ceux où la connaissance est intrinsèquement relationnelle et où les erreurs de diagnostic ont des conséquences financières mesurables) pour lesquels la structure du graphe n'est pas une amélioration de la sémantique vectorielle, mais son remplacement nécessaire. L'intelligence opérationnelle des télécommunications appartient résolument à cette classe.

Bibliographie

- [1] Gary MARCUS et Ernest DAVIS. « How Not to Test GPT-3 ». In : *arXiv preprint arXiv:2212.10529* (2023). DOI : [10.48550/arXiv.2212.10529](https://doi.org/10.48550/arXiv.2212.10529).
- [2] Patrick LEWIS et al. « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 33. 2020, p. 9459-9474.
- [3] A. NEMATBAKHSI et al. « A Survey on Compositional Learning of AI Models: Theoretical and Experimental Practices ». In : *arXiv preprint arXiv:2406.08787* (2024). Comprehensive survey defining compositional reasoning as the ability to understand and manipulate complex structures by decomposing them into simpler parts. URL : <https://arxiv.org/html/2406.08787v1>.
- [4] Linhao LUO et al. « Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning ». In : *arXiv preprint arXiv:2310.01061* (2023). DOI : [10.48550/arXiv.2310.01061](https://doi.org/10.48550/arXiv.2310.01061). URL : <https://arxiv.org/abs/2310.01061>.
- [5] Ziwei Ji et al. « Survey of Hallucination in Natural Language Generation ». In : *ACM Computing Surveys* 55.12 (2023), p. 1-38. DOI : [10.1145/3571730](https://doi.org/10.1145/3571730).
- [6] Hannah KANG et al. « GraphEval: A Knowledge-Graph Based LLM Hallucination Evaluation Framework ». In : *arXiv preprint arXiv:2407.10793* (2024). URL : <https://arxiv.org/pdf/2407.10793.pdf>.
- [7] Ashish VASWANI et al. « Attention Is All You Need ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 30. 2017, p. 5998-6008. URL : <https://arxiv.org/abs/1706.03762>.
- [8] Nelson F. LIU et al. « Lost in the Middle: How Language Models Use Long Contexts ». In : *arXiv preprint arXiv:2307.03172* (2023). Foundational work on positional bias in long contexts. DOI : [10.48550/arXiv.2307.03172](https://doi.org/10.48550/arXiv.2307.03172).
- [9] Vladimir KARPUKHIN et al. « Dense Passage Retrieval for Open-Domain Question Answering ». In : *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, p. 6769-6781. DOI : [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550).
- [10] Michihiro YASUNAGA et al. « QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering ». In : *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 2021, p. 535-546. DOI : [10.18653/v1/2021.naacl-main.45](https://doi.org/10.18653/v1/2021.naacl-main.45).
- [11] Darren EDGE et al. « From Local to Global: A Graph RAG Approach to Query-Focused Summarization ». In : *arXiv preprint arXiv:2404.16130* (2024). Microsoft Research.
- [12] Shirui PAN et al. « Unifying Large Language Models and Knowledge Graphs: A Roadmap ». In : *IEEE Transactions on Knowledge and Data Engineering* 36.7 (2024), p. 3580-3599. DOI : [10.1109/TKDE.2024.3352100](https://doi.org/10.1109/TKDE.2024.3352100).
- [13] Renzo ANGLES et Claudio GUTIERREZ. « An Introduction to Graph Data Management ». In : *arXiv preprint arXiv:1801.00036* (2017). Comprehensive survey of graph database models.
- [14] Ian ROBINSON, Jim WEBBER et Emil EIFREM. *Graph Databases: New Opportunities for Connected Data*. 2nd. O'Reilly Media, 2015. ISBN : 978-1491930892.

- [15] Michael GRIEVES et John VICKERS. « Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems ». In : *Transdisciplinary Perspectives on Complex Systems* (2014), p. 85-113. DOI : [10.1007/978-3-319-38756-7_4](https://doi.org/10.1007/978-3-319-38756-7_4).
- [16] Fei TAO et al. « Digital Twin in Industry: State-of-the-Art ». In : *IEEE Transactions on Industrial Informatics* 15.4 (2019), p. 2405-2415. DOI : [10.1109/TII.2018.2873186](https://doi.org/10.1109/TII.2018.2873186).
- [17] TM FORUM. *Enhanced Telecom Operations Map (eTOM): The Business Process Framework for The Information and Communications Services Industry*. Rapp. tech. GB921 v3.0. Industry reference framework defining OSS and BSS process domains for telecommunications service providers. TM Forum, 2002. URL : <https://www.tmforum.org/open-digital-architecture/process-framework-etom/>.
- [18] Michael B. KELLY. « Report: The TeleManagement Forum's Enhanced Telecom Operations Map (eTOM) ». In : *Journal of Network and Systems Management* 11.1 (2003). Academic overview and analysis of the eTOM framework and its OSS/BSS decomposition., p. 109-119. DOI : [10.1023/A:1022449209526](https://doi.org/10.1023/A:1022449209526).
- [19] Jason WEI et al. « Chain-of-Thought Prompting Elicits Reasoning in Large Language Models ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 35. Foundational paper introducing Chain-of-Thought prompting, demonstrating that intermediate reasoning steps significantly improve LLM performance on complex multi-hop tasks. 2022. URL : <https://arxiv.org/abs/2201.11903>.
- [20] Nelson F. LIU et al. « Lost in the Middle: How Language Models Use Long Contexts ». In : *Transactions of the Association for Computational Linguistics* 12 (2024), p. 157-173. DOI : [10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638).
- [21] Mohammadreza POURREZA et Davood RAFIEI. « DIN-SQL: Context-aware Text-to-SQL Generation ». In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Used to reference schema hallucination dynamics in structured generation tasks. 2023. URL : <https://arxiv.org/abs/2304.11015>.
- [22] Tom B. BROWN et al. « Language Models are Few-Shot Learners ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 33. 2020, p. 1877-1901.
- [23] ANTHROPIC. *Prompt Engineering Interactive Tutorial and Best Practices*. Rapp. tech. Principles of Context Engineering for large context window models. Anthropic, 2024. URL : <https://docs.anthropic.com/en/docs/prompt-engineering>.
- [24] Harrison CHASE. *LangChain*. 2022. URL : <https://github.com/hwchase17/langchain>.
- [25] LANGCHAIN INC. *LangGraph: Building stateful, multi-actor applications with LLMs*. 2024. URL : <https://python.langchain.com/docs/langgraph>.
- [26] META AI. *Introducing Meta Llama 3: The most capable openly available LLM to date*. 2024. URL : <https://ai.meta.com/blog/meta-llama-3/>.
- [27] OPENAI. *Hello GPT-4o*. 2024. URL : <https://openai.com/index/hello-gpt-4o/>.
- [28] Arjun PANICKSSERY, Samuel R. BOWMAN et Shi FENG. « LLM Evaluators Recognize and Favor Their Own Generations ». In : *arXiv preprint arXiv:2404.13076* (2024). URL : <https://arxiv.org/abs/2404.13076>.
- [29] Shahul Es et al. « RAGAS: Automated Evaluation of Retrieval Augmented Generation ». In : *arXiv preprint arXiv:2309.15217* (2023). URL : <https://arxiv.org/abs/2309.15217>.
- [30] Jon SAAD-FALCON et al. « ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems ». In : *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024. URL : <https://arxiv.org/abs/2311.09476>.

- [31] Darren EDGE et al. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. Reference duplication explicitly mapping lowercase citation in latex to the Microsoft GraphRAG study. 2024. arXiv : [2404.16130](https://arxiv.org/abs/2404.16130) [cs.CL]. URL : <https://arxiv.org/abs/2404.16130>.
- [32] Vladimir KARPUKHIN et al. *Dense Passage Retrieval for Open-Domain Question Answering*. 2020. arXiv : [2004.04906](https://arxiv.org/abs/2004.04906) [cs.CL]. URL : <https://arxiv.org/abs/2004.04906>.
- [33] Alex MALLIN et al. « When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories ». In : *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023. URL : <https://arxiv.org/abs/2212.10511>.
- [34] NEO4J. *The Neo4j Cypher Manual: Graph Traversal and Match Syntax*. Neo4j, Inc. 2024. URL : <https://neo4j.com/docs/cypher-manual/current/patterns/>.
- [35] Y. CHEN, H. WANG et M. ZHANG. « Understanding Network Outages in Data Centers: A CMDB Perspective ». In : *IEEE/ACM Transactions on Networking* 27.6 (2019), p. 2451-2464.
- [36] Jay KREPS, Neha NARKHEDE, Jun RAO et al. « Kafka: A Distributed Messaging System for Log Processing ». In : *Proceedings of the NetDB*. T. 11. 2011, p. 1-7.
- [37] Albert Q. JIANG et al. « Mistral 7B ». In : *arXiv preprint arXiv:2310.06825* (2023). URL : <https://arxiv.org/abs/2310.06825>.
- [38] Kai GRESHAKE et al. « More than you've asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models ». In : *arXiv preprint arXiv:2302.12173* (2023). URL : <https://arxiv.org/abs/2302.12173>.

Annexes

Introduction aux Annexes

Note au lecteur : Les extraits de code présentés dans ces annexes ont été expurgés de leur logique utilitaire (gestion des logs, requêtes HTTP, imports standards) afin de mettre en exergue la logique algorithmique fondamentale soutenant les concepts théorisés dans ce mémoire. Le code source intégral du prototype, incluant l'interface conversationnelle et la configuration de déploiement, est disponible sur le dépôt public du projet : [\[https://github.com/Quanghng/telecom-intelligence\]](https://github.com/Quanghng/telecom-intelligence).

Annexe A : Génération du Jumeau Numérique et Injection de Pannes (ChaosMonkey)

Cette annexe démontre la méthodologie de création de la topologie réseau de référence et le mécanisme d'injection de pannes (ChaosMonkey). Ce couplage garantit la reproductibilité absolue des expérimentations (via l'injection déterministe de scénarios pour les mesures de traversée k -hop) tout en simulant le bruit opérationnel d'un réseau réel (via l'injection probabiliste).

A.1 Génération de la topologie de base (Modèle de Barabási-Albert)

Extrait de `src/data_generation/topology.py` illustrant l'instanciation de la couche physique sous forme de graphe sans échelle (scale-free), caractéristique des dorsales de télécommunication.

```
class TopologyBuilder:
    # [...] Initialisation omise pour la clarte

    def build(self) -> nx.Graph:
        """Generate the Barabasi-Albert graph representing the physical layer."""
        self.graph = nx.barabasi_albert_graph(
            n=self.num_nodes,
            m=self.attachment_edges,
            seed=self.seed,
        )

        # Attribution canonique du label de couche physique
        for node in self.graph.nodes:
            self.graph.nodes[node]["layer"] = "physical"

        for u, v in self.graph.edges:
            self.graph.edges[u, v]["layer"] = "physical"

        return self.graph
```

A.2 Le moteur d'injection de pannes (ChaosMonkey)

Extrait de `src/data_generation/chaos.py` démontrant la coexistence de l'injection stochastique (bruit de fond) et de l'amorçage déterministe (Ground Truth) nécessaire à la validation académique des requêtes multi-sauts.

```
class ChaosMonkey:
    # [...] Initialisation omise

    def inject(self, graph: nx.Graph) -> nx.Graph:
        """Apply random faults and deterministic ground-truths to the graph."""
        if self.fault_types:
            graph = self._apply_stochastic_chaos(graph)

        # CRITICAL: Enforce thesis benchmarks after random chaos
        graph = self.enforce_ground_truths(graph)
        return graph

    def _seed_tc_006(self, graph: nx.Graph) -> None:
        """Ensures Hop-3 Ground Truth:
        (service)-[:DEPENDS_ON]->(logical)-[:RUNS_ON]->(physical_Core_Router,'Down')."""
        p_node_id, l_node_id, s_node_id = "SYNTH-CORE-001", "SYNTH-LOGICAL-002",
        "SYNTH-SERVICE-001"

        # 1. Inject Nodes
        graph.add_node(p_node_id, layer="physical", node_type="Core_Router",
            status="Down")
        graph.add_node(l_node_id, layer="logical", node_type="MPLS_Tunnel",
            status="Degraded")
        graph.add_node(s_node_id, layer="service", node_type="Enterprise_VPN",
            status="Degraded")

        # 2. Inject Strict Directed Edges
        graph.add_edge(l_node_id, p_node_id, type="RUNS_ON")
        graph.add_edge(s_node_id, l_node_id, type="DEPENDS_ON")
```

Annexe B : Orchestration Cyclique avec LangGraph

L'architecture présentée repose sur l'abandon des pipelines séquentiels stricts au profit d'une machine à états cyclique. Les extraits de `src/llm/graph_orchestrator.py` ci-dessous prouvent la mise en œuvre de la boucle de rétroaction corrective ($N_{\max} = 3$), permettant au modèle de langage de s'auto-corriger suite à une erreur de syntaxe Cypher ou à une violation du schéma ontologique.

B.1 Définition de l'état du graphe et compilation

```
class CypherGenState(TypedDict):
    user_prompt: str
    retrieved_context: Optional[RetrievedContext]
    generated_cypher: str
    validation_errors: List[str]
```

```

iteration_count: int
is_valid: bool

class CypherGraphOrchestrator:
    # [...] Initialisation omise

    def _build_graph(self) -> Any:
        """Construct and compile the LangGraph StateGraph."""
        workflow = StateGraph(CypherGenState)

        # Ajout des noeuds d'exécution
        workflow.add_node("vector_retrieval", self._node_vector_retrieval)
        workflow.add_node("cypher_generation", self._node_cypher_generation)
        workflow.add_node("validation", self._node_validation)

        # Definition des aretes (flux d'exécution)
        workflow.set_entry_point("vector_retrieval")
        workflow.add_edge("vector_retrieval", "cypher_generation")
        workflow.add_edge("cypher_generation", "validation")

        # Arete conditionnelle : validation -> END (succes) ou -> cypher_generation
        workflow.add_conditional_edges(
            "validation",
            self._route_after_validation,
            {
                "end": END,
                "retry": "cypher_generation",
            },
        )
        return workflow.compile()

```

B.2 Logique de routage conditionnel

```

@staticmethod
def _route_after_validation(state: CypherGenState) -> str:
    """Conditional edge: decide whether to retry or terminate."""
    if state["is_valid"]:
        return "end"

    if state["iteration_count"] >= MAX_ITERATIONS:
        logger.error("Max iterations reached. Graceful failure.")
        return "end"

    logger.info("Validation failed - routing to retry.")
    return "retry"

```

Annexe C : Routage Sémantique et Assemblage Dynamique de Contexte

L'approche GraphRAG développée repose sur l'injection ciblée du schéma ontologique (Dynamic Context Assembly). Plutôt que de saturer la fenêtre de contexte du LLM, le système interroge une base vectorielle

(ChromaDB) pour extraire les requêtes historiques similaires et en déduit l'union mathématique du schéma requis.

C.1 La bibliothèque de requêtes (Few-Shot Knowledge Base)

Extrait de `src/llm/vector_memory.py` montrant la structuration d'une requête de référence (ici un scénario $k = 4$, de la couche physique vers la couche métier).

```
CYPHER_QUERY_LIBRARY: Final[List[Dict[str, Any]]] = [
    # [...] Autres requetes omises
    {
        "id": "ex_05_physical_to_business_full_chain",
        "question": "If Edge_Router 'XYZ' goes 'Down', which customers are affected?",
        "cypher": (
            "MATCH (r:TelecomNode:physical {node_type:'Edge_Router', name:'XYZ'})\n"
            "WHERE r.status='Down'\n"
            "OPTIONAL MATCH\n"
            "(r)-[:RUNS_ON]-(l:TelecomNode:logical)-[:DEPENDS_ON]-(s:TelecomNode:service)-[:SUBSCRIBED_TO]-\n"
            "\n"
            "RETURN r, l, s, c\n"
            "LIMIT 25"
        ),
        "partial_schema": {
            "layers": ["physical", "logical", "service", "business"],
            "node_types": ["Edge_Router"],
            "relationships": ["RUNS_ON", "DEPENDS_ON", "SUBSCRIBED_TO"],
            "properties_used": ["node_type", "name", "status"],
        },
    },
]
```

C.2 Calcul de l'union de schéma (Dynamic Context Assembly)

```
@staticmethod
def _compute_schema_union(examples: List[Dict[str, Any]]) -> Dict[str, Set[str]]:
    """Compute the mathematical set union of partial schemas."""
    union: Dict[str, Set[str]] = {
        "layers": set(),
        "node_types": set(),
        "relationships": set(),
        "properties_used": set(),
    }

    for ex in examples:
        schema = ex.get("partial_schema", {})
        for key in union:
            union[key].update(schema.get(key, []))

    return union
```

Annexe D : Protocole d'Évaluation (Framework RAGAS)

La validité scientifique de l'étude comparative (Vector RAG vs GraphRAG) repose sur une séparation stricte des responsabilités entre le modèle générateur et le modèle évaluateur (LLM-as-a-judge).

D.1 Schéma rigoureux des cas de test (Ground Truth)

Extrait de `src/evaluation/schema.py` utilisant Pydantic pour garantir la conformité typologique des jeux de données de test.

```
class EvaluationTestCase(BaseModel):
    query_id: str = Field(..., description="Unique_test-case_identifier.")
    question: str = Field(..., description="Natural-language_question_posed.")
    ground_truth: str = Field(..., description="Pre-verified_correct_answer.")
    hop_depth: int = Field(..., ge=1, le=5, description="Minimum_relationship_hops_required.")
    category: str = Field(..., description="Semantic_category.")
```

D.2 Orchestration de l'évaluation A/B

Extrait de `run_evaluation.py` illustrant l'instanciation de l'architecture "Dual-Model" pour éviter le biais d'auto-évaluation.

```
def main() -> None:
    # [...] Parsing CLI omis

    # 1. LLM Evaluator (Juge RAGAS : GPT-4o)
    evaluator_llm = ChatOpenAI(model="gpt-4o", temperature=0.0)
    evaluator_embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

    # 2. LLM Generateur (Pipeline GraphRAG : Llama-3.3-70b via Groq)
    openai_client = OpenAI(base_url=_HF_BASE_URL, api_key=api_key)

    # 3. Execution comparative
    runner = EvaluationRunner(
        graph_rag=orchestrator,
        vector_retriever=vector_retriever,
        synthesis_agent=synthesis_agent,
        export_path=raw_csv_path,
    )
    raw_df = runner.run_all(cases)

    # 4. Notation scientifique via RAGAS
    evaluator = RagasEvaluator(
        evaluator_llm=evaluator_llm,
        evaluator_embeddings=evaluator_embeddings
    )
    scored_df = evaluator.evaluate_dataframe(raw_df)

    # Export des resultats pour le Chapitre 4
    scored_df.to_csv(scored_csv_path, index=False)
```